```
/*
* This source file is part of the EtherCAT Slave Stack Code licensed by Beckhoff Automation GmbH & Co KG, 33415 Verl, Germany.
* The corresponding license agreement applies. This hint shall not be removed.
* https://www.beckhoff.com/media/downloads/slave-stack-code/ethercat_ssc_license.pdf
*/

/**
\addtogroup EcatAppl EtherCAT application
@{
*/

/**
\file applInterface.h
\author EthercatSSC@beckhoff.com
\brief Definition of the application interface functions

\version 5.13

<br>Changes to version V5.12:<br>
V5.13 COE4: update default entry name handling in case of 16Bit characters, add CoE Read/write indication functions<br>
<br>Changes to version V5.11:<br>
V5.12 APPL1: add optional application function called from the main loop (after mbx and esm are executed)<br>
V5.12 EEPROM3: implement a store EEPROM timeout handler<br>
V5.12 EOE1: move icmp sample to the sampleappl,add EoE application interface functions<br>
V5.12 FOE1: update new interface,move the FoE sample to sampleappl,add FoE application callback functions<br>
<br>Changes to version - :<br>
V5.10.1 : Start file change log
*/

/*-----------------------------------------------------------------------------------------
------
------    Includes
------
-----------------------------------------------------------------------------------------*/


#ifndef _APPL_INTERFACE_H_
#define _APPL_INTERFACE_H_

#include "ecat_def.h"
#include <stdlib.h>
#include <string.h>
#include "esc.h"
#include "ecatslv.h"
#include "objdef.h"
#include "ecatappl.h"




#endif /*#ifndef _APPL_INTERFACE_H_*/

#if defined(_APPL_INTERFACE_) && (_APPL_INTERFACE_ == 1)
   #define PROTO
```

```
#else
    #define PROTO extern
#endif

/*---------------------------------------------------------------------------------------------
------
------    Function pointer which are used to call optional functions from the application
------
---------------------------------------------------------------------------------------------*/




/* ECATCHANGE_START(V5.13) COE4*/
//////////////////////////////////////////////////////////////////////////////////////////
/**
\param     Index            Index of the object to be read
\param     Subindex         Subindex of the object entry to be read
\param     CompleteAccess   Indicates if the object is read with complete access

\brief    The function is called before an object/entry is read
*//////////////////////////////////////////////////////////////////////////////////////////
PROTO void(*pAPPL_CoeReadInd)(UINT16 Index, UINT8 Subindex, BOOL CompleteAccess);

//////////////////////////////////////////////////////////////////////////////////////////
/**
\param     Index            Index of the object which was written
\param     Subindex         Subindex of the object entry which was written
\param     CompleteAccess   Indicates if the object was written with complete access

\brief    The function is called after an object/entry was successfully written
*//////////////////////////////////////////////////////////////////////////////////////////
PROTO void(*pAPPL_CoeWriteInd)(UINT16 Index, UINT8 Subindex, BOOL CompleteAccess);

/* ECATCHANGE_END(V5.13) COE4*/
//////////////////////////////////////////////////////////////////////////////////////////
/**
\brief    This function is called by the SSC from the main loop
\brief     The function pointer is reset in MainInit() so it shall be set afterwards
*//////////////////////////////////////////////////////////////////////////////////////////
PROTO void(*pAPPL_MainLoop)();

/*---------------------------------------------------------------------------------------------
------
------    Functions provided by the SSC to be called from the application
------
---------------------------------------------------------------------------------------------*/


//////////////////////////////////////////////////////////////////////////////////////////
/**
\return   0 if successful or greater 0 in case of an error
```

```
\brief    Initialize the generic slave stack.
\brief    This function should be called after the platform including operating system and
\brief    ESC is ready to use.
*////////////////////////////////////////////////////////////////////////////////////////
PROTO UINT16 MainInit(void);


////////////////////////////////////////////////////////////////////////////////////////
/**
\brief    This function handles the low priority function like EtherCAT state machine
\brief    handling, mailbox protocols and if no synchronization is enabled also the
\brief    application.
\brief    This function shall be called cyclically from the application.
*////////////////////////////////////////////////////////////////////////////////////////
PROTO void MainLoop(void);


////////////////////////////////////////////////////////////////////////////////////////
/**
\param    alStatus       requested AL Status
\param    alStatusCode   requested AL Status Code

\brief    This function shall be called by the application to trigger state transition in case
\brief    of an application error or to complete a pending transition.
\brief    If the function was called due to an error it shall be again if the error is gone.
\brief    NOTE: state requests to a higher state than the current state are not allowed.
*////////////////////////////////////////////////////////////////////////////////////////
PROTO void ECAT_StateChange(UINT8 alStatus, UINT16 alStatusCode);



#undef PROTO
/*
* This source file is part of the EtherCAT Slave Stack Code licensed by Beckhoff Automation GmbH & Co KG, 33415 Verl, Germany.
* The corresponding license agreement applies. This hint shall not be removed.
* https://www.beckhoff.com/media/downloads/slave-stack-code/ethercat_ssc_license.pdf
*/

/**
 * \addtogroup CiA402appl CiA402 Sample Application
 * @{
 */

/**
\file cia402appl.h
\author EthercatSSC@beckhoff.com
\brief CiA402 specific defines; objects; Axes structure

\version 5.13

<br>Changes to version V5.12:<br>
V5.13 CIA402 1: update names and pdo mapping flags according ETG.6010<br>
<br>Changes to version V5.11:<br>
V5.12 CIA402 1: change define value STATUSWORD_STATE_SWITCHEDONDISABLED<br>
V5.12 CIA402 2: update control word mask for transition 3 (Ready to switch on  to Switched on)<br>
```

```
V5.12 CiA402 3: add 16bit padding to process data of csv and csp mode<br>
<br>Changes to version V5.01:<br>
V5.11 CiA402 1: "append padding byte to ""csv/csp"" process data (basic process data has odd word length), add pack directive"<br>
V5.11 COE1: update invalid end entry in the object dictionaries (error with some compilers)<br>
<br>Changes to version - :<br>
V5.01 : Start file change log
 */

/*---------------------------------------------------------------------------------------------
------
------    Includes
------
---------------------------------------------------------------------------------------------*/
#include "objdef.h"

#ifndef _CIA402_H_
#define _CIA402_H_

/*---------------------------------------------------------------------------------------------
------
------    Defines and Types
------
---------------------------------------------------------------------------------------------*/
/*---------------------------------------------
-    ControlWord Commands Mask (IEC61800_184e)
---------------------------------------------*/
#define CONTROLWORD_COMMAND_SHUTDOWN_MASK                   0x0087 /**< \brief Shutdown command mask*/
#define CONTROLWORD_COMMAND_SWITCHON_MASK                   0x008F /**< \brief Switch on command mask*/
#define CONTROLWORD_COMMAND_SWITCHON_ENABLEOPERATION_MASK   0x008F /**< \brief Switch on & Enable command mask*/
#define CONTROLWORD_COMMAND_DISABLEVOLTAGE_MASK             0x0082 /**< \brief Disable voltage command mask*/
#define CONTROLWORD_COMMAND_QUICKSTOP_MASK                  0x0086 /**< \brief Quickstop command mask*/
#define CONTROLWORD_COMMAND_DISABLEOPERATION_MASK           0x008F /**< \brief Disable operation command mask*/
#define CONTROLWORD_COMMAND_ENABLEOPERATION_MASK            0x008F /**< \brief Enable operation command mask*/
#define CONTROLWORD_COMMAND_FAULTRESET_MASK                 0x0080 /**< \brief Fault reset command mask*/


/*---------------------------------------------
-    ControlWord Commands (IEC61800_184e)
---------------------------------------------*/
/*
 * 0x6040 控制字常用启动顺序：
 *   0x0006：Shutdown，准备上电，目标状态为 Ready To Switch On。
 *   0x0007：Switch On，目标状态为 Switched On。
 *   0x000F：Enable Operation，目标状态为 Operation Enabled。
 *
 * 注意：0x0007 在 Operation Enabled 状态表示 Disable Operation，
 *       会让轴退出运行状态；命令含义必须结合当前状态判断。
 */
#define CONTROLWORD_COMMAND_SHUTDOWN                        0x0006 /**< \brief Shutdown command*/
#define CONTROLWORD_COMMAND_SWITCHON                        0x0007 /**< \brief Switch on command*/
#define CONTROLWORD_COMMAND_SWITCHON_ENABLEOPERATION        0x000F /**< \brief Switch on & Enable command*/
#define CONTROLWORD_COMMAND_DISABLEVOLTAGE                  0x0000 /**< \brief Disable voltage command*/
#define CONTROLWORD_COMMAND_QUICKSTOP                       0x0002 /**< \brief Quickstop command*/
#define CONTROLWORD_COMMAND_DISABLEOPERATION                0x0007 /**< \brief Disable operation command*/
#define CONTROLWORD_COMMAND_ENABLEOPERATION                 0x000F /**< \brief Enable operation command*/
```

```
#define CONTROLWORD_COMMAND_FAULTRESET                      0x0080 /**< \brief Fault reset command*/


/*---------------------------------------------
-    StatusWord Masks and Flags
---------------------------------------------*/
#define STATUSWORD_STATE_MASK                               0x006F /**< \brief State mask*/
#define STATUSWORD_VOLTAGE_ENABLED                          0x0010 /**< \brief Indicate high voltage enabled*/
#define STATUSWORD_WARNING                                  0x0080 /**< \brief Warning active*/
#define STATUSWORD_MANUFACTORSPECIFIC                       0x0100 /**< \brief Manufacturer specific*/
#define STATUSWORD_INTERNAL_LIMIT                           0x0800 /**< \brief Internal limit*/
#define STATUSWORD_REMOTE                                   0x0200 /**< \brief Set if the control word is processed*/
#define STATUSWORD_TARGET_REACHED                           0x0400 /**< \brief Target reached*/
#define STATUSWORD_INTERNALLIMITACTIVE                      0x0800 /**< \brief Internal limit active*/
#define STATUSWORD_DRIVE_FOLLOWS_COMMAND                    0x1000 /**< \brief Drive follows command (used in cyclic synchronous
modes)*/
#define STATUSWORD_HOMING_ERROR                             0x2000 /**< \brief Homing error (used in homing mode)*/


/*---------------------------------------------
-    StatusWord
---------------------------------------------*/
/*
 * 主站判断 0x6041 状态时应先执行：status_word & 0x006F。
 * 启动过程中常见结果：
 *   0x0021：Ready To Switch On，控制字 0x0006 已处理。
 *   0x0023：Switched On，控制字 0x0007 已处理。
 *   0x0027：Operation Enabled，控制字 0x000F 已处理。
 * 完整状态字还可能包含 0x0200 Remote、0x0010 Voltage Enabled 等位。
 */
#define STATUSWORD_STATE_NOTREADYTOSWITCHON                 0x0000 /**< \brief Not ready to switch on*/
#define STATUSWORD_STATE_SWITCHEDONDISABLED                 0x0040 /**< \brief Switched on but disabled*/
#define STATUSWORD_STATE_READYTOSWITCHON                    0x0021 /**< \brief Ready to switch on*/
#define STATUSWORD_STATE_SWITCHEDON                         0x0023 /**< \brief Switched on*/
#define STATUSWORD_STATE_OPERATIONENABLED                   0x0027 /**< \brief Operation enabled*/
#define STATUSWORD_STATE_QUICKSTOPACTIVE                    0x0007 /**< \brief Quickstop active*/
#define STATUSWORD_STATE_FAULTREACTIONACTIVE                0x000F /**< \brief Fault reaction active*/
#define STATUSWORD_STATE_FAULT                              0x0008 /**< \brief Fault state*/


/*---------------------------------------------
-    CiA402 State machine
---------------------------------------------*/
/*
 * 以下数值只供 MCU 内部变量 i16State 使用，不是主站读取的 0x6041 值。
 * 例如内部 STATE_OPERATION_ENABLED 为 0x0010，主站状态值为 0x0027。
 */
#define STATE_NOT_READY_TO_SWITCH_ON        0x0001 /**< \brief Not ready to switch on (optional)*/
#define STATE_SWITCH_ON_DISABLED            0x0002 /**< \brief Switch on but disabled (optional)*/
#define STATE_READY_TO_SWITCH_ON            0x0004 /**< \brief Ready to switch on (mandatory)*/
#define STATE_SWITCHED_ON                   0x0008 /**< \brief Switch on (mandatory)*/
#define STATE_OPERATION_ENABLED             0x0010 /**< \brief Operation enabled (mandatory)*/
#define STATE_QUICK_STOP_ACTIVE             0x0020 /**< \brief Quick stop active (optional)*/
#define STATE_FAULT_REACTION_ACTIVE         0x0040 /**< \brief Fault reaction active (mandatory)*/
#define STATE_FAULT                         0x0080 /**< \brief Fault state (mandatory)*/
```

```
/*---------------------------------------------
-    CiA402 Modes of Operation (object 0x6060) (IEC61800_184e)
---------------------------------------------*/
// -128 to -1 Manufacturer-specific operation modes
#define NO_MODE                     0 /**< \brief No mode*/
#define PROFILE_POSITION_MODE       1 /**< \brief Position Profile mode*/
#define VELOCITY_MODE               2 /**< \brief Velocity mode*/
#define PROFILE_VELOCITY_MODE       3 /**< \brief Velocity Profile mode*/
#define PROFILE_VELOCITY_MOCE       PROFILE_VELOCITY_MODE /**< \brief Backward-compatible alias*/
#define PROFILE_TORQUE_MODE         4 /**< \brief Torque Profile mode*/
//5 reserved
#define HOMING_MODE                 6 /**< \brief Homing mode*/
#define INTERPOLATION_POSITION_MODE 7 /**< \brief Interpolation Position mode*/
#define CYCLIC_SYNC_POSITION_MODE   8 /**< \brief Cyclic Synchronous Position mode*/
#define CYCLIC_SYNC_VELOCITY_MODE   9 /**< \brief Cyclic Synchronous Velocity mode*/
#define CYCLIC_SYNC_TORQUE_MODE     10/**< \brief Cyclic Synchronous Torque mode*/
//+11 to +127 reserved


/************************************
 CiA402 Error Codes (object 0x603F) (IEC61800_184e)
 ************************************/
#define ERROR_SHORT_CIRCUIT_EARTH_LEAKAGE_INPUT             0x2110 /**< \brief Short circuit/earth leakage (input)*/
#define ERROR_EARTH_LEAKAGE_INPUT                           0x2120 /**< \brief Earth leakage (input)*/
#define ERROR_EARTH_LEAKAGE_PHASE_L1                        0x2121 /**< \brief Earth leakage phase L1*/
#define ERROR_EARTH_LEAKAGE_PHASE_L2                        0x2122 /**< \brief Earth leakage phase L2*/
#define ERROR_EARTH_LEAKAGE_PHASE_L3                        0x2123 /**< \brief Earth leakage phase L3*/
#define ERROR_SHORT_CIRCUIT_INPUT                           0x2130 /**< \brief Short circuit (input)*/
#define ERROR_SHORT_CIRCUIT_PHASES_L1_L2                    0x2131 /**< \brief Short circuit phases L1-L2*/
#define ERROR_SHORT_CIRCUIT_PHASES_L2_L3                    0x2132 /**< \brief Short circuit phases L2-L3*/
#define ERROR_SHORT_CIRCUIT_PHASES_L3_L1                    0x2133 /**< \brief Short circuit phases L3-L1*/
#define ERROR_INTERNAL_CURRENT_NO1                          0x2211 /**< \brief Internal current no 1*/
#define ERROR_INTERNAL_CURRENT_NO2                          0x2212 /**< \brief Internal current no 2*/
#define ERROR_OVER_CURRENT_IN_RAMP_FUNCTION                 0x2213 /**< \brief Over-current in ramp function*/
#define ERROR_OVER_CURRENT_IN_THE_SEQUENCE                  0x2214 /**< \brief Over-current in the sequence*/
#define ERROR_CONTINUOUS_OVER_CURRENT_DEVICE_INTERNAL       0x2220 /**< \brief Continuous over current (device internal)*/
#define ERROR_CONTINUOUS_OVER_CURRENT_DEVICE_INTERNAL_NO1   0x2221 /**< \brief Continuous over current no 1*/
#define ERROR_CONTINUOUS_OVER_CURRENT_DEVICE_INTERNAL_NO2   0x2222 /**< \brief Continuous over current no 2*/
#define ERROR_SHORT_CIRCUIT_EARTH_LEAKAGE_DEVICE_INTERNAL   0x2230 /**< \brief Short circuit/earth leakage (device internal)*/
#define ERROR_EARTH_LEAKAGE_DEVICE_INTERNAL                 0x2240 /**< \brief Earth leakage (device internal)*/
#define ERROR_SHORT_CIRCUIT_DEVICE_INTERNAL                 0x2250 /**< \brief Short circuit (device internal)*/
#define ERROR_CONTINUOUS_OVER_CURRENT                       0x2310 /**< \brief Continuous over current*/
#define ERROR_CONTINUOUS_OVER_CURRENT_NO1                   0x2311 /**< \brief Continuous over current no 1*/
#define ERROR_CONTINUOUS_OVER_CURRENT_NO2                   0x2312 /**< \brief Continuous over current no 2*/
#define ERROR_SHORT_CIRCUIT_EARTH_LEAKAGE_MOTOR_SIDE        0x2320 /**< \brief Short circuit/earth leakage (motor-side)*/
#define ERROR_EARTH_LEAKAGE_MOTOR_SIDE                      0x2330 /**< \brief Earth leakage (motor-side)*/
#define ERROR_EARTH_LEAKAGE_PHASE_U                         0x2331 /**< \brief Earth leakage phase U*/
#define ERROR_EARTH_LEAKAGE_PHASE_V                         0x2332 /**< \brief Earth leakage phase V*/
#define ERROR_EARTH_LEAKAGE_PHASE_W                         0x2333 /**< \brief Earth leakage phase W*/
#define ERROR_SHORT_CIRCUIT_MOTOR_SIDE                      0x2340 /**< \brief Short circuit (motor-side)*/
#define ERROR_SHORT_CIRCUIT_PHASES_U_V                      0x2341 /**< \brief Short circuit phases U-V*/
#define ERROR_EARTH_LEAKAGE_PHASE_V_W                       0x2342 /**< \brief Earth leakage phase V-W*/
#define ERROR_EARTH_LEAKAGE_PHASE_W_U                       0x2343 /**< \brief Earth leakage phase W-U*/
#define ERROR_LOAD_LEVEL_FAULT_I2T_THERMAL_STATE            0x2350 /**< \brief Load level fault (I2t, thermal state)*/
```

```
#define ERROR_LOAD_LEVEL_WARNING_I2T_THERMAL_STATE        0x2351 /**< \brief Load level warning (I2t, thermal state)*/
#define ERROR_MAINS_OVER_VOLTAGE                          0x3110 /**< \brief Mains over-voltage*/
#define ERROR_MAINS_OVER_VOLTAGE_PHASE_L1                 0x3111 /**< \brief Mains over-voltage phase L1*/
#define ERROR_MAINS_OVER_VOLTAGE_PHASE_L2                 0x3112 /**< \brief Mains over-voltage phase L2 */
#define ERROR_MAINS_OVER_VOLTAGE_PHASE_L3                 0x3113 /**< \brief Mains over-voltage phase L3*/
#define ERROR_MAINS_UNDER_VOLTAGE                         0x3120 /**< \brief Mains under-voltage*/
#define ERROR_MAINS_UNDER_VOLTAGE_PHASE_L1                0x3121 /**< \brief Mains under-voltage phase L1*/
#define ERROR_MAINS_UNDER_VOLTAGE_PHASE_L2                0x3122 /**< \brief Mains under-voltage phase L2*/
#define ERROR_MAINS_UNDER_VOLTAGE_PHASE_L3                0x3123 /**< \brief Mains under-voltage phase L3*/
#define ERROR_PHASE_FAILURE                               0x3130 /**< \brief Phase failure*/
#define ERROR_PHASE_FAILURE_L1                            0x3131 /**< \brief Phase failure L1*/
#define ERROR_PHASE_FAILURE_L2                            0x3132 /**< \brief Phase failure L2*/
#define ERROR_PHASE_FAILURE_L3                            0x3133 /**< \brief Phase failure L3*/
#define ERROR_PHASE_SEQUENCE                              0x3134 /**< \brief Phase sequence*/
#define ERROR_MAINS_FREQUENCY                             0x3140 /**< \brief Mains frequency*/
#define ERROR_MAINS_FREQUENCY_TOO_GREAT                   0x3141 /**< \brief Mains frequency too great*/
#define ERROR_MAINS_FREQUENCY_TOO_SMALL                   0x3142 /**< \brief Mains frequency too small*/
#define ERROR_DC_LINK_OVER_VOLTAGE                        0x3210 /**< \brief DC link over-voltage*/
#define ERROR_OVER_VOLTAGE_NO_1                           0x3211 /**< \brief Over-voltage no  1*/
#define ERROR_OVER_VOLTAGE_NO_2                           0x3212 /**< \brief Over voltage no  2 */
#define ERROR_DC_LINK_UNDER_VOLTAGE                       0x3220 /**< \brief DC link under-voltage*/
#define ERROR_UNDER_VOLTAGE_NO_1                          0x3221 /**< \brief Under-voltage no  1*/
#define ERROR_UNDER_VOLTAGE_NO_2                          0x3222 /**< \brief Under-voltage no  2*/
#define ERROR_LOAD_ERROR                                  0x3230 /**< \brief Load error*/
#define ERROR_OUTPUT_OVER_VOLTAGE                         0x3310 /**< \brief Output over-voltage*/
#define ERROR_OUTPUT_OVER_VOLTAGE_PHASE_U                 0x3311 /**< \brief Output over-voltage phase U*/
#define ERROR_OUTPUT_OVER_VOLTAGE_PHASE_V                 0x3312 /**< \brief Output over-voltage phase V*/
#define ERROR_OUTPUT_OVER_VOLTAGE_PHASE_W                 0x3313 /**< \brief Output over-voltage phase W*/
#define ERROR_ARMATURE_CIRCUIT                            0x3320 /**< \brief Armature circuit*/
#define ERROR_ARMATURE_CIRCUIT_INTERRUPTED                0x3321 /**< \brief Armature circuit interrupted*/
#define ERROR_FIELD_CIRCUIT                               0x3330 /**< \brief Field circuit error */
#define ERROR_FIELD_CIRCUIT_INTERRUPTED                   0x3331 /**< \brief Field circuit interrupted*/
#define ERROR_EXCESS_AMBIENT_TEMPERATURE                  0x4110 /**< \brief Excess ambient temperature*/
#define ERROR_TOO_LOW_AMBIENT_TEMPERATURE                 0x4120 /**< \brief Too low ambient temperature*/
#define ERROR_TEMPERATURE_SUPPLY_AIR                      0x4130 /**< \brief Temperature supply air*/
#define ERROR_TEMPERATURE_AIR_OUTLET                      0x4140 /**< \brief Temperature air outlet*/
#define ERROR_EXCESS_TEMPERATURE_DEVICE                   0x4210 /**< \brief Excess temperature device*/
#define ERROR_TOO_LOW_TEMPERATURE_DEVICE                  0x4220 /**< \brief Too low temperature device*/
#define ERROR_TEMPERATURE_DRIVE                           0x4300 /**< \brief Temperature drive error*/
#define ERROR_EXCESS_TEMPERATURE_DRIVE                    0x4310 /**< \brief Excess temperature drive error*/
#define ERROR_TOO_LOW_TEMPERATURE_DRIVE                   0x4320 /**< \brief Too low temperature drive error*/
#define ERROR_TEMPERATURE_SUPPLY                          0x4400 /**< \brief Temperature supply error*/
#define ERROR_EXCESS_TEMPERATURE_SUPPLY                   0x4410 /**< \brief Excess temperature supply*/
#define ERROR_TOO_LOW_TEMPERATURE_SUPPLY                  0x4420 /**< \brief Too low temperature supply*/
#define ERROR_SUPPLY_ERROR                                0x5100 /**< \brief Supply error*/
#define ERROR_SUPPLY_LOW_VOLTAGE                          0x5110 /**< \brief Supply low voltage*/
#define ERROR_U1_SUPPLY_15V                               0x5111 /**< \brief U1 = supply +15V/-15V*/
#define ERROR_U2_SUPPLY_24_V                              0x5112 /**< \brief U2 = supply +24 V*/
#define ERROR_U3_SUPPLY_5_V                               0x5113 /**< \brief U3 = supply +5 V*/
#define ERROR_U4_MANUFACTURER_SPECIFIC                    0x5114 /**< \brief U4 = manufacturer-specific error*/
#define ERROR_U5_MANUFACTURER_SPECIFIC                    0x5115 /**< \brief U5 = manufacturer-specific error*/
#define ERROR_U6_MANUFACTURER_SPECIFIC                    0x5116 /**< \brief U6 = manufacturer-specific error*/
#define ERROR_U7_MANUFACTURER_SPECIFIC                    0x5117 /**< \brief U7 = manufacturer-specific error*/
#define ERROR_U8_MANUFACTURER_SPECIFIC                    0x5118 /**< \brief U8 = manufacturer-specific error*/
#define ERROR_U9_MANUFACTURER_SPECIFIC                    0x5119 /**< \brief U9 = manufacturer-specific error*/
```

```
#define ERROR_SUPPLY_INTERMEDIATE_CIRCUIT                   0x5120 /**< \brief Supply intermediate circuit*/
//#define ERROR_CONTROL                                     0x5200
#define ERROR_CONTROL_MEASUREMENT_CIRCUIT                   0x5210 /**< \brief Measurement circuit*/
#define ERROR_CONTROL_COMPUTING_CIRCUIT                     0x5220 /**< \brief Computing circuit*/
#define ERROR_OPERATING_UNIT                                0x5300 /**< \brief Operating unit error*/
#define ERROR_POWER_SECTION                                 0x5400 /**< \brief Power section error*/
#define ERROR_OUTPUT_STAGES                                 0x5410 /**< \brief Output stages error*/
#define ERROR_CHOPPER                                       0x5420 /**< \brief Chopper error*/
#define ERROR_INPUT_STAGES                                  0x5430 /**< \brief Input stages error*/
#define ERROR_CONTACTS_ERROR                                0x5440 /**< \brief Contacts error*/
#define ERROR_CONTACT_1_MANUFACTURER_SPECIFIC               0x5441 /**< \brief Contact 1 = manufacturer-specific error*/
#define ERROR_CONTACT_2_MANUFACTURER_SPECIFIC               0x5442 /**< \brief Contact 2 = manufacturer-specific error*/
#define ERROR_CONTACT_3_MANUFACTURER_SPECIFIC               0x5443 /**< \brief Contact 3 = manufacturer-specific error*/
#define ERROR_CONTACT_4_MANUFACTURER_SPECIFIC               0x5444 /**< \brief Contact 4 = manufacturer-specific error*/
#define ERROR_CONTACT_5_MANUFACTURER_SPECIFIC               0x5445 /**< \brief Contact 5 = manufacturer-specific error*/
#define ERROR_FUSES_ERROR                                   0x5450 /**< \brief Fuses error*/
#define ERROR_S1_L1                                         0x5451 /**< \brief S1 = l1 error*/
#define ERROR_S2_L2                                         0x5452 /**< \brief S2 = l2 error*/
#define ERROR_S3_L3                                         0x5453 /**< \brief S3 = l3 error*/
#define ERROR_S4_MANUFACTURER_SPECIFIC                      0x5454 /**< \brief S4 = manufacturer-specific error*/
#define ERROR_S5_MANUFACTURER_SPECIFIC                      0x5455 /**< \brief S5 = manufacturer-specific error*/
#define ERROR_S6_MANUFACTURER_SPECIFIC                      0x5456 /**< \brief S6 = manufacturer-specific error*/
#define ERROR_S7_MANUFACTURER_SPECIFIC                      0x5457 /**< \brief S7 = manufacturer-specific error*/
#define ERROR_S8_MANUFACTURER_SPECIFIC                      0x5458 /**< \brief S8 = manufacturer-specific error*/
#define ERROR_S9_MANUFACTURER_SPECIFIC                      0x5459 /**< \brief S9 = manufacturer-specific error*/
#define ERROR_HARDWARE_MEMORY                               0x5500 /**< \brief Hardware memory error*/
#define ERROR_RAM                                           0x5510 /**< \brief RAM error*/
#define ERROR_ROM_EPROM                                     0x5520 /**< \brief ROM/EPROM error*/
#define ERROR_EEPROM                                        0x5530 /**< \brief EEPROM error*/
#define ERROR_SOFTWARE_RESET_WATCHDOG                       0x6010 /**< \brief Software reset (watchdog)*/
//0x6301_TO_0x630F        ERROR_DATA_RECORD_NO_1_TO_NO_15
#define ERROR_LOSS_OF_PARAMETERS                            0x6310 /**< \brief Loss of parameters*/
#define ERROR_PARAMETER_ERROR                               0x6320 /**< \brief Parameter error*/
#define ERROR_POWER_ERROR                                   0x7100 /**< \brief Power error*/
#define ERROR_BRAKE_CHOPPER                                 0x7110 /**< \brief Brake chopper*/
#define ERROR_FAILURE_BRAKE_CHOPPER                         0x7111 /**< \brief Failure brake chopper*/
#define ERROR_OVER_CURRENT_BRAKE_CHOPPER                    0x7112 /**< \brief Over current brake chopper*/
#define ERROR_PROTECTIVE_CIRCUIT_BRAKE_CHOPPER              0x7113 /**< \brief Protective circuit brake chopper error*/
#define ERROR_MOTOR_ERROR                                   0x7120 /**< \brief Motor error*/
#define ERROR_MOTOR_BLOCKED                                 0x7121 /**< \brief Motor blocked error*/
#define ERROR_MOTOR_ERROR_OR_COMMUTATION_MALFUNC            0x7122 /**< \brief Motor error or commutation malfunc */
#define ERROR_MOTOR_TILTED                                  0x7123 /**< \brief Motor tilted*/
#define ERROR_MEASUREMENT_CIRCUIT                           0x7200 /**< \brief Measurement circuit*/
#define ERROR_SENSOR_ERROR                                  0x7300 /**< \brief Sensor error*/
#define ERROR_TACHO_FAULT                                   0x7301 /**< \brief Tacho fault*/
#define ERROR_TACHO_WRONG_POLARITY                          0x7302 /**< \brief Tacho wrong polarity*/
#define ERROR_RESOLVER_1_FAULT                              0x7303 /**< \brief Resolver 1 fault*/
#define ERROR_RESOLVER_2_FAULT                              0x7304 /**< \brief Resolver 2 fault*/
#define ERROR_INCREMENTAL_SENSOR_1_FAULT                    0x7305 /**< \brief Incremental sensor 1 fault*/
#define ERROR_INCREMENTAL_SENSOR_2_FAULT                    0x7306 /**< \brief Incremental sensor 2 fault*/
#define ERROR_INCREMENTAL_SENSOR_3_FAULT                    0x7307 /**< \brief Incremental sensor 3 fault*/
#define ERROR_SPEED                                         0x7310 /**< \brief Speed error*/
#define ERROR_POSITION                                      0x7320 /**< \brief Position error*/
#define ERROR_COMPUTATION_CIRCUIT                           0x7400 /**< \brief Computation circuit*/
#define ERROR_COMMUNICATION                                 0x7500 /**< \brief Communication error*/
```

```
#define ERROR_SERIAL_INTERFACE_NO_1                     0x7510 /**< \brief Serial interface no  1 error*/
#define ERROR_SERIAL_INTERFACE_NO_2                     0x7520 /**< \brief Serial interface no  2 error*/
#define ERROR_DATA_STORAGE_EXTERNAL                     0x7600 /**< \brief Data storage (external) error*/
#define ERROR_TORQUE_CONTROL                            0x8300 /**< \brief Torque control error*/
#define ERROR_EXCESS_TORQUE                             0x8311 /**< \brief Excess torque error*/
#define ERROR_DIFFICULT_START_UP                        0x8312 /**< \brief Difficult start up error*/
#define ERROR_STANDSTILL_TORQUE                         0x8313 /**< \brief Standstill torque error*/
#define ERROR_INSUFFICIENT_TORQUE                       0x8321 /**< \brief Insufficient torque error*/
#define ERROR_TORQUE_FAULT                              0x8331 /**< \brief Torque fault*/
#define ERROR_VELOCITY_SPEED_CONTROLLER                 0x8400 /**< \brief Velocity speed controller*/
#define ERROR_POSITION_CONTROLLER                       0x8500 /**< \brief Position controller*/
#define ERROR_POSITIONING_CONTROLLER                    0x8600 /**< \brief Positioning controller*/
#define ERROR_FOLLOWING_ERROR                           0x8611 /**< \brief Following error*/
#define ERROR_REFERENCE_LIMIT                           0x8612 /**< \brief Reference limit*/
#define ERROR_SYNC_CONTROLLER                           0x8700 /**< \brief Sync controller*/
#define ERROR_WINDING_CONTROLLER                        0x8800 /**< \brief Winding controller*/
#define ERROR_PROCESS_DATA_MONITORING                   0x8900 /**< \brief Process data monitoring*/
//#define ERROR_CONTROL                                 0x8A00
#define ERROR_DECELERATION                              0xF001 /**< \brief Deceleration error*/
#define ERROR_SUB_SYNCHRONOUS_RUN                       0xF002 /**< \brief Sub-synchronous run error*/
#define ERROR_STROKE_OPERATION                          0xF003 /**< \brief Stroke operation error*/
//#define ERROR_CONTROL                                 0xF004
//0xFF00_TO_0xFFFF        MANUFACTURER_SPECIFIC


/*---------------------------------------------
-    CiA402 generic error option code values
       Note: Not all values are valid for each error option code.
       A detailed description of the option code values are listed in the specification IEC 61800-7-200
       0x605B    : action in state transition 8
       0x605C    : action in state transition 5
---------------------------------------------*/
#define DISABLE_DRIVE                   0 /**< \brief Disable drive (options: 0x605B; 0x605C; 0x605E)*/
#define SLOW_DOWN_RAMP                  1 /**< \brief Slow down ramp (options: 0x605B; 0x605C; 0x605E)*/
#define QUICKSTOP_RAMP                  2 /**< \brief Quick stop ramp (options: 0x605E)*/
#define STOP_ON_CURRENT_LIMIT           3 /**< \brief Stop on current limit (options: 0x605E)*/
#define STOP_ON_VOLTAGE_LIMIT           4 /**< \brief Stop on voltage limit (options: 0x605E)*/


/*---------------------------------------------
-    Specific values for Quick stop option code (object 0x605A) (IEC61800_184e)
       indicated the quick stop function
---------------------------------------------*/
//-32768 to -1        MANUFACTURER_SPECIFIC
#define SLOWDOWN_RAMP_NO_TRANSIT                5 /**< \brief Slow down on slow down ramp and stay in Quick Stop Active*/
#define QUICKSTOP_RAMP_NO_TRANSIT               6 /**< \brief Slow down on quick stop ramp and stay in Quick Stop Active*/
#define CURRENT_LIMIT_NO_TRANSIT                7 /**< \brief Slow down on current limit and stay in Quick Stop Active*/
#define VOLTAGE_LIMIT_NO_TRANSIT                8 /**< \brief Slow down on voltage limit and stay in Quick Stop Active*/
//9 to 32767        RESERVED


#define MAX_AXES    1 /**< \brief This carrier drives one physical motor axis*/
```

```
/*-----------------------------------------------
-    Module Identifications (each supported operation mode has a module)
-----------------------------------------------*/
#define CSV_CSP_MODULE_ID       0x00119800 /**< \brief Module Id for cyclic synchronous position/velocity mode (dynamic switching
supported)*/
#define CSP_MODULE_ID           0x00219800 /**< \brief Module Id for cyclic synchronous position mode*/
#define CSV_MODULE_ID           0x00319800 /**< \brief Module Id for cyclic synchronous velocity mode*/


/**
 * \addtogroup PdoMappingObjects PDO Mapping Objects
 * csp/csv mode RxPDO mapping : 0x1600<br>
 * csp     mode RxPDO mapping : 0x1601<br>
 * csv     mode RxPDO mapping : 0x1602<br>
 * csp/csv mode TxPDO mapping : 0x1A00<br>
 * csp     mode TxPDO mapping : 0x1A01<br>
 * csv     mode TxPDO mapping : 0x1A02
 * @{
 */
/** \brief 0x1600 (csp/csv RxPDO) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
   UINT32   aEntries[5]; /**< \brief Entry buffer*/
} OBJ_STRUCT_PACKED_END
TOBJ1600;


/** \brief 0x1601 (csp RxPDO) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
   UINT32   aEntries[3]; /**< \brief Entry buffer*/
} OBJ_STRUCT_PACKED_END
TOBJ1601;


/** \brief 0x1602 (csv RxPDO) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
   UINT32   aEntries[3]; /**< \brief Entry buffer*/
} OBJ_STRUCT_PACKED_END
TOBJ1602;


/** \brief 0x1A00 (csp/csv TxPDO) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
   UINT32   aEntries[5]; /**< \brief Entry buffer*/
} OBJ_STRUCT_PACKED_END
TOBJ1A00;


/** \brief 0x1A01 (csp TxPDO) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
   UINT32   aEntries[3]; /**< \brief Entry buffer*/
} OBJ_STRUCT_PACKED_END
```

```
TOBJ1A01;


/** \brief 0x1A02 (csv TxPDO) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
   UINT32   aEntries[5]; /**< \brief Entry buffer*/
} OBJ_STRUCT_PACKED_END
TOBJ1A02;
/** @}*/


/**
 * \addtogroup SmAssignObjects SyncManager Assignment Objects
 * SyncManager 2 : 0x1C12<br>
 * SyncManager 3 : 0x1C13
 * @{
 */
/** \brief 0x1C12 (SyncManager 2 assignment) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
   UINT16   aEntries[MAX_AXES]; /**< \brief Entry buffer (number of entries depends on the supported axis, each axis requires one
entry)*/
} OBJ_STRUCT_PACKED_END
TOBJ1C12;


/** \brief 0x1C13 (SyncManager 3 assignment) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
   UINT16   aEntries[MAX_AXES]; /**< \brief Entry buffer (number of entries depends on the supported axis, each axis requires one
entry)*/
} OBJ_STRUCT_PACKED_END
TOBJ1C13;
/** @}*/


/**
 * \addtogroup CiA402Objects CiA402 Axis Objects
 * @{
 */
/** \brief 0x607D (Software Position Limit) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
   INT32    i32MinLimit; /**< \brief Minimum limit*/
   INT32    i32MaxLimit; /**< \brief Maximum limit*/
} OBJ_STRUCT_PACKED_END
TOBJ607D;


/** \brief Object 0x60C2 (Interpolation Time Period) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
   UINT8    u8InterpolationPeriod; /**< \brief Interpolation period*/
   INT8     i8InterpolationIndex; /**< \brief Interpolation index*/
} OBJ_STRUCT_PACKED_END
TOBJ60C2;
```

```
/** @}*/


/**
 * \addtogroup DeviceParaObjects Device Parameter Objects
 * Modular Device Profile : 0xF000<br>
 * Module Profile List : 0xF010<br>
 * Configured Modules Ident List : 0xF030<br>
 * Detected Modules Ident List : 0xF050<br>
 * @{
 */
/** \brief 0xF000 (Modular Device Profile) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex0*/
   UINT16   u16Moduleindexdistance; /**< \brief Module Index distance
                                     *
                                     * Index distance between two modules.<br>
                                     * According to ETG.6010 the object index distance is 0x800*/
   UINT16   u16Maximumnumberofmodules; /**< \brief Maximum number of modules*/
} OBJ_STRUCT_PACKED_END
TOBJF000;


/** \brief 0xF010 (Module Profile List) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex0*/
   UINT32   aEntries[MAX_AXES];/**< \brief Module profile information buffer
                                *
                                * Bit 00..15: Profile number of the module on position 1<br>
                                * Bit 16..31: Profile specific<br>
                                * The number of entries depends on the number of supported axes.
                                */
} OBJ_STRUCT_PACKED_END
TOBJF010;


/** \brief 0xF030 (Configured Modules Ident List) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex0*/
   UINT32   aEntries[MAX_AXES];/**< \brief Module Ident List buffer
                                * The number of entries depends on the number of supported axes.
                                */
} OBJ_STRUCT_PACKED_END
TOBJF030;


/** \brief 0xF050 (Detected Modules Ident List) data structure*/
typedef struct OBJ_STRUCT_PACKED_START {
   UINT16   u16SubIndex0; /**< \brief SubIndex0*/
   UINT32   aEntries[MAX_AXES];/**< \brief Module Ident List buffer
                                * The number of entries depends on the number of supported axes.
                                */
} OBJ_STRUCT_PACKED_END
TOBJF050;
/** @}*/
```

```
/*---------------------------------------------
-    Generic Axis structure
---------------------------------------------*/
/**
 * \addtogroup PDO Process Data Objects
 * @{
 */
/** \brief Data structure to handle the process data transmitted via 0x1A00 (csp/csv TxPDO)*/
typedef __packed struct STRUCT_PACKED_START
{
    UINT16 ObjStatusWord; /**< \brief Status word (0x6041)*/
    INT32 ObjPositionActualValue; /**< \brief Actual position (0x6064)*/
    INT32 ObjVelocityActualValue; /**< \brief Actual velocity (0x606C)*/
    INT16 ObjModesOfOperationDisplay; /**< \brief Current mode of operation (0x6061)*/
}STRUCT_PACKED_END
TCiA402PDO1A00;


/** \brief Data structure to handle the process data transmitted via 0x1A01 (csp TxPDO)*/
typedef struct STRUCT_PACKED_START
{
    UINT16 ObjStatusWord; /**< \brief Status word (0x6041)*/
    INT32 ObjPositionActualValue; /**< \brief Actual position (0x6064)*/
    UINT16 Padding16Bit; /**< \brief 16bit padding*/
}STRUCT_PACKED_END
TCiA402PDO1A01;


/** \brief Data structure to handle the process data transmitted via 0x1A02 (csv TxPDO)*/
typedef __packed struct STRUCT_PACKED_START
{
    UINT16 ObjStatusWord; /**< \brief Status word (0x6041)*/
    INT32 ObjPositionActualValue; /**< \brief Actual position (0x6064)*/
    INT32 ObjVelocityActualValue; /**< \brief Actual velocity (0x606C)*/
    INT16 ObjModesOfOperationDisplay; /**< \brief Mode display plus 8bit padding*/
}STRUCT_PACKED_END
TCiA402PDO1A02;


/** \brief Data structure to handle the process data transmitted via 0x1600 (csp/csv RxPDO)*/
typedef __packed struct STRUCT_PACKED_START
{
    UINT16 ObjControlWord; /**< \brief Control word (0x6040)*/
    INT32 ObjTargetPosition; /**< \brief Target position (0x607A)*/
    INT32 ObjTargetVelocity; /**< \brief Target velocity (0x60FF)*/
    INT16 ObjModesOfOperation; /**< \brief Mode of operation (0x6060)*/
}STRUCT_PACKED_END
TCiA402PDO1600;


/** \brief Data structure to handle the process data transmitted via 0x1601 (csp RxPDO)*/
typedef __packed struct STRUCT_PACKED_START
{
```

```
    UINT16 ObjControlWord; /**< \brief Control word (0x6040)*/
    INT32 ObjTargetPosition; /**< \brief Target position (0x607A)*/
    UINT16 Padding16Bit; /**< \brief 16bit padding*/
}STRUCT_PACKED_END
TCiA402PDO1601;


/** \brief Data structure to handle the process data transmitted via 0x1602 (csv RxPDO)*/
typedef __packed struct STRUCT_PACKED_START
{
    UINT16 ObjControlWord; /**< \brief Control word (0x6040)*/
    INT32 ObjTargetVelocity; /**< \brief Target velocity (0x60FF)*/
    UINT16 Padding16Bit; /**< \brief 16bit padding*/
}STRUCT_PACKED_END
TCiA402PDO1602;
/** @}*/



/**
 * \addtogroup CiA402Objects CiA402 Axis Objects
 * @{
 */
/** \brief Data structure to handle the axis specific object data*/
typedef struct OBJ_STRUCT_PACKED_START
{
    TOBJ1600 sRxPDOMap0; /**< \brief csv/csp RxPDO (0x1600)*/
    TOBJ1601 sRxPDOMap1; /**< \brief csp RxPDO (0x1601)*/
    TOBJ1602 sRxPDOMap2; /**< \brief csv RxPDO (0x1602)*/
    TOBJ1A00 sTxPDOMap0; /**< \brief csv/csp TxPDO (0x1A00)*/
    TOBJ1A01 sTxPDOMap1; /**< \brief csp TxPDO (0x1A01)*/
    TOBJ1A02 sTxPDOMap2; /**< \brief csv TxPDO (0x1A02)*/

    UINT16 objErrorCode; /**< \brief Error Code (0x603F)*/
    UINT16 objControlWord; /**< \brief Control Word (0x6040)*/
    UINT16 objStatusWord; /**< \brief Status Word (0x6041)*/
    INT16 objQuickStopOptionCode; /**< \brief Quick Stop Option Code (0x605A)*/
    INT16 objShutdownOptionCode; /**< \brief Shutdown Option Code (0x605B)*/
    INT16 objDisableOperationOptionCode; /**< \brief Disable Operation Option Code (0x605C)*/
    INT16 objFaultReactionCode; /**< \brief Fault Reaction Code (0x605E)*/
    INT16 objModesOfOperation; /**< \brief Modes of Operation (0x6060)*/
    INT16 objModesOfOperationDisplay; /**< \brief Mode of Operation Display (0x6061)*/
    INT32 objPositionActualValue; /**< \brief Position Actual Value (0x6064)*/
    INT32 objVelocityActualValue; /**< \brief Actual Velocity Value (0x606C)*/
    INT16 objTorqueActualValue; /**< \brief Torque Actual Value (0x6077)*/
    INT32 objTargetPosition; /**< \brief Target Position (0x607A)*/
    TOBJ607D objSoftwarePositionLimit; /**< \brief Software Position limit (0x607D)*/
    UINT32 objProfileVelocity; /**< \brief Profile Velocity (0x6081)*/
    UINT32 objProfileAcceleration; /**< \brief Profile Acceleration (0x6083)*/
    UINT32 objProfileDeceleration; /**< \brief Profile Deceleration (0x6084)*/
    UINT32 objQuickStopDeclaration; /**< \brief Quick Stop Declaration (0x6085)*/
    TOBJ60C2 objInterpolationTimePeriod; /**< \brief Interpolation Time Period (0x60C2)*/
    INT32 objTargetVelocity; /**< \brief Target Velocity (0x60FF)*/
    UINT32 objSupportedDriveModes; /**< \brief Supported Drive Modes (0x6502)*/
    UINT16 objMotionUnits; /* 0x2000: 0=legacy rpm, 1=count/s; PREOP only */
```

```
    INT16 objHaltOptionCode; /* 0x605D: 1=profile ramp, 2=quick-stop ramp */
    UINT32 objMaxProfileVelocity; /* 0x607F */
    INT32 objPositionOffset; /* 0x60B0: CSP target offset */
}OBJ_STRUCT_PACKED_END
CiA402Objects;



#define CIA402_OBJECTS_SIZE     SIZEOF(CiA402Objects) /**< \brief Size of CiA402 object structure*/


/** \brief Data structure to handle an Cia402 axis*/
typedef struct
{
    BOOL        bAxisIsActive; /**< \brief Indicates if active is active*/
    BOOL        bBrakeApplied; /**< \brief Motor break is applied*/
    BOOL        bLowLevelPowerApplied; /**< \brief Low Level Power applied*/
    BOOL        bHighLevelPowerApplied; /**< \brief High Level Power applied*/
    BOOL        bAxisFunctionEnabled; /**< \brief Axis functions enabled*/
    BOOL        bConfigurationAllowed; /**< \brief Configuration allowed*/

    UINT16      i16State; /**< \brief Axis state*/
    UINT16      u16PendingOptionCode; /**< \brief Pending operation code*/
    double      fCurPosition; /**< \brief Current position within control loop*/
    UINT32      u32CycleTime; /**< \brief Motion controller cycletime in us*/


    CiA402Objects        Objects; /**< \brief CiA402 Axis object variable*/
    TOBJECT OBJMEM        * ObjDic; /**< \brief Pointer to Axis specific object dictionary*/
}
TCiA402Axis, *pTCiA402Axis;
/** @}*/

#endif //_CIA402_H_

#ifdef _CiA402_
UINT8 WriteMotionParameter(UINT16 index, UINT8 subindex, UINT32 dataSize,
                           UINT16 MBXMEM *pData, UINT8 bCompleteAccess);
OBJCONST TSDOINFOENTRYDESC OBJMEM sEntryDesc0x2000 =
    {DEFTYPE_UNSIGNED16, 16, ACCESS_READ | ACCESS_WRITE_PREOP};
OBJCONST UCHAR OBJMEM aName0x2000[] = "Motion units (0=rpm, 1=count/s)";
OBJCONST TSDOINFOENTRYDESC OBJMEM sEntryDesc0x605D =
    {DEFTYPE_INTEGER16, 16, ACCESS_READWRITE};
OBJCONST UCHAR OBJMEM aName0x605D[] = "Halt option code";
OBJCONST TSDOINFOENTRYDESC OBJMEM sEntryDesc0x607F =
    {DEFTYPE_UNSIGNED32, 32, ACCESS_READWRITE};
OBJCONST UCHAR OBJMEM aName0x607F[] = "Max profile velocity";
OBJCONST TSDOINFOENTRYDESC OBJMEM sEntryDesc0x60B0 =
    {DEFTYPE_INTEGER32, 32, ACCESS_READWRITE | OBJACCESS_RXPDOMAPPING};
OBJCONST UCHAR OBJMEM aName0x60B0[] = "Position offset (CSP)";
    #define PROTO
#else
    #define PROTO extern
#endif
```

```
/*---------------------------------------------
-    Entry Descriptions and object names
---------------------------------------------*/
#ifdef _CiA402_
/**
 * \addtogroup SmAssignObjects SyncManager Assignment Objects
 * @{
 */

/**
 * \brief Entry descriptions of SyncManager assign objects
 *
 * SubIndex0<br>
 * SubIndex1 (for all other entries the same description will be used (because the object code is ARRAY))
 */
OBJCONST TSDOINFOENTRYDESC    OBJMEM asPDOAssignEntryDesc[] = {
   {DEFTYPE_UNSIGNED8, 0x08, (ACCESS_READ|ACCESS_WRITE_PREOP)},
   {DEFTYPE_UNSIGNED16, 0x10, (ACCESS_READ|ACCESS_WRITE_PREOP)}};


/**
 * \brief Object 0x1C12 (Sync Manager 2 assign) object and entry names
 *
 * In this example no specific entry name is defined ("SubIndex xxx" is used)
 */
OBJCONST UCHAR OBJMEM aName0x1C12[] = "RxPDO assign";

/**
 * \brief Object 0x1C13 (Sync Manager 3 assign) object and entry names
 *
 * In this example no specific entry name is defined ("SubIndex xxx" is used)
 */
OBJCONST UCHAR OBJMEM aName0x1C13[] = "TxPDO assign";
/** @}*/


/**
* \addtogroup PdoMappingObjects PDO Mapping Objects
* @{
*/
/**
 * \brief Object 0x1600 (csp/csv RxPDO) entry descriptions
 */
OBJCONST TSDOINFOENTRYDESC  OBJMEM asEntryDesc0x1600[] = {
   {DEFTYPE_UNSIGNED8, 0x8, (ACCESS_READ|ACCESS_WRITE_PREOP) }, /* Subindex 000 */
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 001*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 002*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 003*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 004*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}}; /* SubIndex 005*/


/**
 * \brief Object 0x1600 (csp/csv RxPDO) object and entry names
```

```
 *
 * In this example no specific entry name is defined ("SubIndex xxx" is used)
 */
OBJCONST UCHAR OBJMEM aName0x1600[] = "csp/csv RxPDO\000\377";


/**
 * \brief Object 0x1601 (csp RxPDO) entry descriptions
 */
OBJCONST TSDOINFOENTRYDESC  OBJMEM asEntryDesc0x1601[] = {
   {DEFTYPE_UNSIGNED8, 0x8, (ACCESS_READ|ACCESS_WRITE_PREOP) }, /* Subindex 000 */
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 001*/
   { DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP) }, /* SubIndex 002*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}}; /* SubIndex 003*/


/**
 * \brief Object 0x1601 (csp RxPDO) object and entry names
 *
 * In this example no specific entry name is defined ("SubIndex xxx" is used)
 */
OBJCONST UCHAR OBJMEM aName0x1601[] = "csp RxPDO\000\377";


/**
 * \brief Object 0x1602 (csv RxPDO) entry descriptions
 */
OBJCONST TSDOINFOENTRYDESC  OBJMEM asEntryDesc0x1602[] = {
   {DEFTYPE_UNSIGNED8, 0x8, (ACCESS_READ|ACCESS_WRITE_PREOP) }, /* Subindex 000 */
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 001*/
   { DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP) }, /* SubIndex 002*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}}; /* SubIndex 003*/


/**
 * \brief Object 0x1602 (csv RxPDO) object and entry names
 *
 * In this example no specific entry name is defined ("SubIndex xxx" is used)
 */
OBJCONST UCHAR OBJMEM aName0x1602[] = "csv RxPDO\000\377";



/**
 * \brief Object 0x1A00 (csp/csv TxPDO) entry descriptions
 */
OBJCONST TSDOINFOENTRYDESC  OBJMEM asEntryDesc0x1A00[] = {
   {DEFTYPE_UNSIGNED8, 0x8, (ACCESS_READ|ACCESS_WRITE_PREOP) }, /* Subindex 000 */
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 001*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 002*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 003*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 004*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}}; /* SubIndex 005 */
```

```
/**
 * \brief Object 0x1A00 (csp/csv TxPDO) object and entry names
 *
 * In this example no specific entry name is defined ("SubIndex xxx" is used)
 */
OBJCONST UCHAR OBJMEM aName0x1A00[] = "csp/csv TxPDO\000\377";


/**
 * \brief Object 0x1A01 (csp RxPDO) entry descriptions
 */
OBJCONST TSDOINFOENTRYDESC  OBJMEM asEntryDesc0x1A01[] = {
   {DEFTYPE_UNSIGNED8, 0x8, (ACCESS_READ|ACCESS_WRITE_PREOP) }, /* Subindex 000 */
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 001*/
   { DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP) }, /* SubIndex 002*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}}; /* SubIndex 003 */


/**
 * \brief Object 0x1A01 (csp TxPDO) object and entry names
 *
 * In this example no specific entry name is defined ("SubIndex xxx" is used)
 */
OBJCONST UCHAR OBJMEM aName0x1A01[] = "csp TxPDO\000\377";



/**
 * \brief Object 0x1A02 (csv RxPDO) entry descriptions
 */
OBJCONST TSDOINFOENTRYDESC  OBJMEM asEntryDesc0x1A02[] = {
   {DEFTYPE_UNSIGNED8, 0x8, (ACCESS_READ|ACCESS_WRITE_PREOP) }, /* Subindex 000 */
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 001*/
   { DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP) }, /* SubIndex 002*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 003*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}, /* SubIndex 004*/
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}}; /* SubIndex 005*/


/**
 * \brief Object 0x1A02 (csv TxPDO) object and entry names
 *
 * In this example no specific entry name is defined ("SubIndex xxx" is used)
 */
OBJCONST UCHAR OBJMEM aName0x1A02[] = "csv TxPDO\000\377";
/** @}*/



/**
 * \addtogroup CiA402Objects CiA402 Axis Objects
 * @{
 */
 /*ECATCHANGE_START(V5.13) CIA402 1*/
```

```
/** \brief Object 0x603F (Error Code) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x603F = {DEFTYPE_UNSIGNED16, 0x10, (ACCESS_READ | OBJACCESS_TXPDOMAPPING) };

/** \brief Object 0x603F (Error Code) object name*/
OBJCONST UCHAR OBJMEM aName0x603F[] = "Error Code";

/** \brief Object 0x6040 (Control Word) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x6040 = {DEFTYPE_UNSIGNED16, 0x10, (ACCESS_READWRITE|OBJACCESS_RXPDOMAPPING)};

/** \brief Object 0x6040 (Control Word) object name*/
OBJCONST UCHAR OBJMEM aName0x6040[] = "Control Word";

/** \brief Object 0x6041 (Status Word) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x6041 = {DEFTYPE_UNSIGNED16, 0x10, (ACCESS_READ|OBJACCESS_TXPDOMAPPING)};

/** \brief Object 0x6041 (Status Word) object name*/
OBJCONST UCHAR OBJMEM aName0x6041[] = "Status Word";

/** \brief Object 0x605A (Quickstop Option Code) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x605A = {DEFTYPE_INTEGER16, 0x10, ACCESS_READWRITE};

/** \brief Object 0x605A (Quickstop Option Code) object name*/
OBJCONST UCHAR OBJMEM aName0x605A[] = "Quick stop option code";

/** \brief Object 0x605B (Shutdown Option Code) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x605B = {DEFTYPE_INTEGER16, 0x10, ACCESS_READWRITE};

/** \brief Object 0x605B (Shutdown Option Code) object name*/
OBJCONST UCHAR OBJMEM aName0x605B[] = "Shutdown Option Code";


/** \brief Object 0x605C (Disable Operation Option Code) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x605C = {DEFTYPE_INTEGER16, 0x10, ACCESS_READWRITE};

/** \brief Object 0x605C (Disable Operation Option Code) object name*/
OBJCONST UCHAR OBJMEM aName0x605C[] = "Disable Operation Option Code";

/** \brief Object 0x605E (Fault Reaction Code) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x605E = {DEFTYPE_INTEGER16, 0x10, ACCESS_READWRITE};

/** \brief Object 0x605E (Fault Reaction Code) object name*/
OBJCONST UCHAR OBJMEM aName0x605E[] = "Fault reaction option code";

/** \brief Object 0x6060 (Modes of Operation) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x6060 = {DEFTYPE_INTEGER8, 0x08, (ACCESS_READWRITE|OBJACCESS_RXPDOMAPPING)};

/** \brief Object 0x6060 (Modes of Operation) object name*/
OBJCONST UCHAR OBJMEM aName0x6060[] = "Modes of Operation";

/** \brief Object 0x6061 (Modes of Operation Display) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x6061 = {DEFTYPE_INTEGER8, 0x08, (ACCESS_READ | OBJACCESS_TXPDOMAPPING)};

/** \brief Object 0x6061 (Modes of Operation Display) object name*/
OBJCONST UCHAR OBJMEM aName0x6061[] = "Modes of Operation Display";
```

```
/** \brief Object 0x6064 (Position Actual Value) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x6064 = {DEFTYPE_INTEGER32, 0x20, (ACCESS_READ | OBJACCESS_TXPDOMAPPING)};

/** \brief Object 0x6064 (Position Actual Value) object name*/
OBJCONST UCHAR OBJMEM aName0x6064[] = "Position Actual Value";

/** \brief Object 0x606C (Velocity Actual Value) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x606C = {DEFTYPE_INTEGER32, 0x20, (ACCESS_READ | OBJACCESS_TXPDOMAPPING)};

/** \brief Object 0x606C (Velocity Actual Value) object name*/
OBJCONST UCHAR OBJMEM aName0x606C[] = "Velocity Actual Value";

/** \brief Object 0x6077 (Torque Actual Value) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x6077 = {DEFTYPE_INTEGER16, 0x10, (ACCESS_READ | OBJACCESS_TXPDOMAPPING) };

/** \brief Object 0x6077 (Torque Actual Value) object name*/
OBJCONST UCHAR OBJMEM aName0x6077[] = "Torque Actual Value";

/** \brief Object 0x607A (Target Position) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x607A = {DEFTYPE_INTEGER32, 0x20, (ACCESS_READWRITE | OBJACCESS_RXPDOMAPPING)};

/** \brief Object 0x607A (Target Position) object name*/
OBJCONST UCHAR OBJMEM aName0x607A[] = "Target Position";

/**
 * \brief Object 0x607D (Software Position Limit) entry description
 *
 * Subindex 0<br>
 * Subindex 1 : Min Position range limit<br>
 * Subindex 2 : Max Position range limit
 */
OBJCONST TSDOINFOENTRYDESC    OBJMEM asEntryDesc0x607D[] = {
    {DEFTYPE_UNSIGNED8, 0x08, ACCESS_READ},
    {DEFTYPE_INTEGER32, 0x20, (ACCESS_READWRITE | OBJACCESS_RXPDOMAPPING)},
    {DEFTYPE_INTEGER32, 0x20, (ACCESS_READWRITE | OBJACCESS_RXPDOMAPPING)}};


/** \brief Object 0x607D (Software Position Limit) object and entry names*/
OBJCONST UCHAR OBJMEM aName0x607D[] = "Software Position Limit\000Min position limit\000Max position limit\000\377";


/** \brief Object 0x6081 (Profile Velocity) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x6081 = {DEFTYPE_UNSIGNED32, 0x20, ACCESS_READWRITE};

/** \brief Object 0x6081 (Profile Velocity) object name*/
OBJCONST UCHAR OBJMEM aName0x6081[] = "Profile Velocity";


/** \brief Object 0x6083 (Profile Acceleration) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x6083 = {DEFTYPE_UNSIGNED32, 0x20, ACCESS_READWRITE};

/** \brief Object 0x6083 (Profile Acceleration) object name*/
OBJCONST UCHAR OBJMEM aName0x6083[] = "Profile Acceleration";
```

```
/** \brief Object 0x6084 (Profile Deceleration) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x6084 = {DEFTYPE_UNSIGNED32, 0x20, ACCESS_READWRITE};

/** \brief Object 0x6084 (Profile Deceleration) object name*/
OBJCONST UCHAR OBJMEM aName0x6084[] = "Profile Deceleration";


/** \brief Object 0x6085 (Quickstop Declaration) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x6085 = {DEFTYPE_UNSIGNED32, 0x20, ACCESS_READWRITE};

/** \brief Object 0x6085 (Quickstop Declaration) object name*/
OBJCONST UCHAR OBJMEM aName0x6085[] = "Quick stop deceleration";


/**
 * \brief Object 0x60C2 (Interpolation Time Period) entry description
 *
 * Subindex 0<br>
 * Subindex 1 : Interpolation time period value<br>
 * Subindex 2 : Interpolation time index
 */
OBJCONST TSDOINFOENTRYDESC    OBJMEM asEntryDesc0x60C2[] = {
    {DEFTYPE_UNSIGNED8, 0x08, ACCESS_READ},
    {DEFTYPE_UNSIGNED8, 0x08, (ACCESS_READWRITE | OBJACCESS_RXPDOMAPPING)},
    {DEFTYPE_INTEGER8, 0x08, (ACCESS_READWRITE | OBJACCESS_RXPDOMAPPING)}};

/** \brief Object 0x60C2 (Interpolation Time Period) object and entry names*/
OBJCONST UCHAR OBJMEM aName0x60C2[] = "Interpolation Time Period\000Interpolation time period value\000Interpolation time
index\000\377";


/** \brief Object 0x60FF (Target Velocity) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x60FF = {DEFTYPE_INTEGER32, 0x20, (ACCESS_READWRITE | OBJACCESS_RXPDOMAPPING)};

/** \brief Object 0x60FF (Target Velocity) object name*/
OBJCONST UCHAR OBJMEM aName0x60FF[] = "Target Velocity";


/** \brief Object 0x6502 (Supported Drive Modes) entry description*/
OBJCONST TSDOINFOENTRYDESC    OBJMEM sEntryDesc0x6502 = {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ | OBJACCESS_TXPDOMAPPING )};

/*ECATCHANGE_END(V5.13) CIA402 1*/
/** \brief Object 0x6502 (Supported Drive Modes) object name*/
OBJCONST UCHAR OBJMEM aName0x6502[] = "Supported Drive Modes";
/** @}*/


/**
 * \addtogroup DeviceParaObjects Device Parameter Objects
 * @{
 */
/**
 * \brief Object 0xF000 (Modular Device Profile) entry description
 *
 * Subindex 000 <br>
 * SubIndex 001: Module index distance<br>
```

```
 * SubIndex 002: Maximum number of modules
 */
OBJCONST TSDOINFOENTRYDESC    OBJMEM asEntryDesc0xF000[3] = {
   {DEFTYPE_UNSIGNED8, 0x8, ACCESS_READ },
   {DEFTYPE_UNSIGNED16, 0x10, ACCESS_READ},
   {DEFTYPE_UNSIGNED16, 0x10, ACCESS_READ}};

/** \brief Object 0xF000 (Modular Device Profile) object and entry names*/
OBJCONST UCHAR OBJMEM aName0xF000[] = "Modular Device Profile\000Index distance\000Maximum number of modules\000\377";


/**
 * \brief 0xF010 (Module profile list) entry descriptions
 *
 * Subindex 0<br>
 * SubIndex x
 * (x > 0)
 */
OBJCONST TSDOINFOENTRYDESC    OBJMEM asEntryDesc0xF010[3] = {
   {DEFTYPE_UNSIGNED8, 0x08, ACCESS_READ},
   {DEFTYPE_UNSIGNED32, 0x20, ACCESS_READ},
   {DEFTYPE_UNSIGNED32, 0x20, ACCESS_READ}};

/**
 * \brief 0xF010 (Module profile list) object name
 *
 * no entry names defined because the object is an array (for every entry > 0 "SubIndex xxx" is returned)
 */
OBJCONST UCHAR OBJMEM aName0xF010[] = "Module Profile List";


/**
 * \brief 0xF030 (Configured Modules Ident) entry descriptions
 *
 * Subindex 0<br>
 * SubIndex x
 * (x > 0)
 */
OBJCONST TSDOINFOENTRYDESC    OBJMEM asEntryDesc0xF030[] = {
   {DEFTYPE_UNSIGNED8, 0x08, (ACCESS_READ|ACCESS_WRITE_PREOP)},
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)},
   {DEFTYPE_UNSIGNED32, 0x20, (ACCESS_READ|ACCESS_WRITE_PREOP)}};

/**
 * \brief 0xF030 (Configured Modules Ident) object name
 *
 * no entry names defined because the object is an array (for every entry > 0 "SubIndex xxx" is returned)
 */
OBJCONST UCHAR OBJMEM aName0xF030[] = "Configured Module Ident List";


/**
 * \brief 0xF050 (Module detected list) entry descriptions
 *
 * Subindex 0<br>
```

```
 * SubIndex x
 * (x > 0)
 */
OBJCONST TSDOINFOENTRYDESC    OBJMEM asEntryDesc0xF050[] = {
   {DEFTYPE_UNSIGNED8, 0x08, ACCESS_READ},
   {DEFTYPE_UNSIGNED32, 0x20, ACCESS_READ},
   {DEFTYPE_UNSIGNED32, 0x20, ACCESS_READ}};


/**
 * \brief 0xF050 (Module detected list) object name
 *
 * no entry names defined because the object is an array (for every entry > 0 "SubIndex xxx" is returned)
 */
OBJCONST UCHAR OBJMEM aName0xF050[] = "Module detected list";
/** @}*/
#endif


/**
 * \addtogroup SmAssignObjects SyncManager Assignment Objects
 * @{
 */

/**
 * \brief 0x1C12 (SyncManager 2 assignment) variable to handle object data
 *
 * SubIndex 0 : 1<br>
 * SubIndex 1 : 0x1600
 * SubIndex 2 : 0
 */
PROTO TOBJ1C12 sRxPDOassign
#ifdef _CiA402_
= {1,{0x1600}}
#endif
;


/**
 * \brief 0x1C13 (SyncManager 3 assignment) variable to handle object data
 *
 * SubIndex 0 : 1<br>
 * SubIndex 1 : 0x1A00<br>
 * SubIndex 2 : 0
 */
PROTO TOBJ1C13 sTxPDOassign
#ifdef _CiA402_
= {1,{0x1A00}}
#endif
;
/** @}*/


/**
 * \addtogroup DeviceParaObjects Device Parameter Objects
```

```
 * @{
 */
/**
 * \brief Object 0xF000 (Modular Device Profile) variable to handle object data
 *
 * SubIndex 0 : 2<br>
 * SubIndex 1 : 0x800 (index distance)<br>
 * SubIndex 2 : MAX_AXES (number of modules depends on the number of axes)
 */
PROTO TOBJF000 sModulardeviceprofile
#ifdef _CiA402_
= {2, 0x800, MAX_AXES}
#endif
;


/**
 * \brief Object 0xF010 (Module profile list) variable to handle object data
 *
 * SubIndex 0 : 1<br>
 * SubIndex 1 :2 (Low Word)<br>
 * SubIndex 2 : 0
 */
PROTO TOBJF010 sModuleProfileInfo
#ifdef _CiA402_
= {1,{2}}
#endif
;

/**
 * \brief Object 0xF030 (Configured Module List) variable to handle object data
 *
 * SubIndex 0 : 1<br>
 * SubIndex 1 : combined csp/csv module is set by default<br>
 * SubIndex 2 : 0
 */
PROTO TOBJF030 sConfiguredModuleIdentList
#ifdef _CiA402_
= {1,{CSV_CSP_MODULE_ID}}
#endif
;

PROTO UINT8 Write0xF030( UINT16 index, UINT8 subindex, UINT32 dataSize, UINT16 MBXMEM * pData, UINT8 bCompleteAccess );


/**
 * \brief Object 0xF050 (Detected Module List) variable to handle object data
 *
 * SubIndex 0 : 1<br>
 * SubIndex 1 : combined csp/csv module is set by default<br>
 * SubIndex 2 : 0
 */
PROTO TOBJF050 sDetectedModuleIdentList
#ifdef _CiA402_
= {1,{CSV_CSP_MODULE_ID}}
```

```
#endif
;
/** @}*/


/**
 * \addtogroup CiA402Objects CiA402 Axis Objects
 * @{
 */
/**
 * \brief Axis objects default values
 *
 */
PROTO CiA402Objects DefCiA402ObjectValues
#ifdef _CiA402_
= {
{5, {0x60400010,0x607A0020,0x60FF0020,0x60600008,0x00000008}},    /* TOBJ1600*/
{3, {0x60400010,0x607A0020,0x00000010 }}, /*TOBJ1601*/
{3, {0x60400010,0x60FF0020,0x00000010 }}, /*TOBJ1602*/
{5, {0x60410010,0x60640020,0x606C0020,0x60610008,0x00000008}}, /*TOBJ1A00*/
{3, {0x60410010,0x60640020,0x00000010 }}, /*TOBJ1A01*/
{5, {0x60410010,0x60640020,0x606C0020,0x60610008,0x00000008}},/*TOBJ1A02*/
0x0,/*(UINT16) ErrorCode 0x1001*/
0x0,/*(UINT16) ControlWord 0x6040*/
0x0,/*(UINT16) StatusWord 0x6041*/
0x2,/*(INT16) QuickStopOptionCode 0x605A*/
DISABLE_DRIVE,/*(INT16)ShutdownOptionCode 0x605B*/
SLOW_DOWN_RAMP, /*(INT16) DisableOperationCode 0x605C*/
QUICKSTOP_RAMP,/*(INT16) FaultReactionCode 0x605E*/
CYCLIC_SYNC_VELOCITY_MODE,/*(INT16) ModeOfOperation 0x6060*/
CYCLIC_SYNC_VELOCITY_MODE,/*(INT16) Mode Of Operation Display 0x6061*/
0x0,/*(INT32) Position Actual Value 0x6064*/
0x0,/*(INT32) Velocity Actual Value 0x606C*/
0x0,/*(INT16) Torque Actual Value 0x6077*/
0x0,/*(INT32) Target Position 0x607A*/
{2,0x88CA6C00,0x77359400},/*TOBJ607D Software Position Limit (minLimit: -2000000000 / maxLimit: 2000000000)*/
100,/*(UINT32) Profile Velocity 0x6081, rpm*/
300,/*(UINT32) Profile Acceleration 0x6083, rpm/s*/
300,/*(UINT32) Profile Deceleration 0x6084, rpm/s*/
300,/*(UINT32) Quick stop deceleration 0x6085, rpm/s*/
{2,1,(INT8)-3},/*TOBJ60C2 Interpolation Time Period*/
0x0,/*(INT32) Target Velocity    0x60FF*/
0x185,/*(UINT32) Supported Drive Modes 0x6502: PP/PV/CSP/CSV */
0, /* motion units: backwards-compatible rpm */
1, /* Halt uses profile deceleration */
1000, /* maximum profile velocity, rpm */
0 /* CSP position offset */
}
#endif
;
/** @}*/


/**
 *\brief Base CiA402 device object dictionary
```

```
 */
PROTO TOBJECT    OBJMEM ApplicationObjDic[]
#ifdef _CiA402_
= {/* Object 0x1C12 */
   {NULL,NULL, 0x1C12, {DEFTYPE_UNSIGNED16, MAX_AXES | (OBJCODE_ARR << 8)}, asPDOAssignEntryDesc, aName0x1C12, &sRxPDOassign, NULL,
NULL, 0x0000 },
   /* Object 0x1C13 */
   {NULL,NULL, 0x1C13, {DEFTYPE_UNSIGNED16, MAX_AXES | (OBJCODE_ARR << 8)}, asPDOAssignEntryDesc, aName0x1C13, &sTxPDOassign, NULL,
NULL, 0x0000 },
    /* Object 0xF000 */
   {NULL,NULL,   0xF000, {DEFTYPE_RECORD, 2 | (OBJCODE_REC << 8)}, asEntryDesc0xF000, aName0xF000, &sModulardeviceprofile, NULL,
NULL, 0x0000 },
    /* Object 0xF010 */
   {NULL,NULL,   0xF010, {DEFTYPE_UNSIGNED32, MAX_AXES | (OBJCODE_ARR << 8)}, asEntryDesc0xF010, aName0xF010, &sModuleProfileInfo,
NULL, NULL, 0x0000 },
    /* Object 0xF030 */
   {NULL,NULL,   0xF030, {DEFTYPE_UNSIGNED32, MAX_AXES | (OBJCODE_ARR << 8)}, asEntryDesc0xF030, aName0xF030,
&sConfiguredModuleIdentList, NULL, Write0xF030, 0x0000 },
   /* Object 0xF050 */
   {NULL,NULL, 0xF050, {DEFTYPE_UNSIGNED32, MAX_AXES | (OBJCODE_ARR << 8)}, asEntryDesc0xF050, aName0xF050,
&sDetectedModuleIdentList, NULL, NULL, 0x0000 },
   {NULL,NULL, 0xFFFF, {0, 0}, NULL, NULL, NULL, NULL, NULL, 0x000}}
#endif
;



/**
 *\brief Object dictionary related to on CiA402 Axis
 */
PROTO TOBJECT    OBJMEM DefCiA402AxisObjDic[]
#ifdef _CiA402_
= {
    /* Object 0x1600 */
    {NULL,NULL, 0x1600, {DEFTYPE_PDOMAPPING, 5 | (OBJCODE_REC << 8)}, asEntryDesc0x1600, aName0x1600, NULL, NULL, NULL, 0x0000 },
    /* Object 0x1601 */
   {NULL,NULL, 0x1601, {DEFTYPE_PDOMAPPING, 3 | (OBJCODE_REC << 8)}, asEntryDesc0x1601, aName0x1601,NULL, NULL, NULL, 0x0000 },
    /* Object 0x1602 */
   {NULL,NULL, 0x1602, {DEFTYPE_PDOMAPPING, 3 | (OBJCODE_REC << 8)}, asEntryDesc0x1602, aName0x1602, NULL, NULL, NULL, 0x0000 },
   /* Object 0x1A00 */
   {NULL,NULL, 0x1A00, {DEFTYPE_PDOMAPPING, 5 | (OBJCODE_REC << 8)}, asEntryDesc0x1A00, aName0x1A00, NULL, NULL, NULL, 0x0000 },
   /* Object 0x1A01 */
   {NULL,NULL, 0x1A01, {DEFTYPE_PDOMAPPING, 3 | (OBJCODE_REC << 8)}, asEntryDesc0x1A01, aName0x1A01, NULL, NULL, NULL, 0x0000 },
   /* Object 0x1A02 */
   {NULL,NULL, 0x1A02, {DEFTYPE_PDOMAPPING, 5 | (OBJCODE_REC << 8)}, asEntryDesc0x1A02, aName0x1A02, NULL, NULL, NULL, 0x0000 },
   /* Object 0x603F */
   {NULL,NULL, 0x603F, {DEFTYPE_UNSIGNED16 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x603F, aName0x603F, NULL, NULL, NULL, 0x0000 },
   /* Object 0x6040 */
   {NULL,NULL, 0x6040, {DEFTYPE_UNSIGNED16 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x6040, aName0x6040, NULL, NULL, NULL, 0x0000 },
   /* Object 0x6041 */
   {NULL,NULL, 0x6041, {DEFTYPE_UNSIGNED16 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x6041, aName0x6041, NULL, NULL, NULL, 0x0000 },
   /* Object 0x605A */
    {NULL,NULL, 0x605A, {DEFTYPE_INTEGER16 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x605A, aName0x605A, NULL, NULL,
WriteMotionParameter, 0x0000 },
   /* Object 0x605B */
    {NULL,NULL, 0x605B, {DEFTYPE_INTEGER16 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x605B, aName0x605B, NULL, NULL,
WriteMotionParameter, 0x0000 },
   /* Object 0x605C */
    {NULL,NULL, 0x605C, {DEFTYPE_INTEGER16 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x605C, aName0x605C, NULL, NULL,
WriteMotionParameter, 0x0000 },
    {NULL,NULL, 0x605D, {DEFTYPE_INTEGER16, OBJCODE_VAR << 8}, &sEntryDesc0x605D, aName0x605D, NULL, NULL, WriteMotionParameter, 0},
   /* Object 0x605E */
    {NULL,NULL, 0x605E, {DEFTYPE_INTEGER16 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x605E, aName0x605E, NULL, NULL,
WriteMotionParameter, 0x0000 },
   /* Object 0x6060 */
```

```
    {NULL,NULL, 0x6060, {DEFTYPE_INTEGER8 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x6060, aName0x6060, NULL, NULL, NULL, 0x0000 },
   /* Object 0x6061 */
   {NULL,NULL, 0x6061, {DEFTYPE_INTEGER8 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x6061, aName0x6061, NULL, NULL, NULL, 0x0000 },
   /* Object 0x6064 */
   {NULL,NULL, 0x6064, {DEFTYPE_INTEGER32 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x6064, aName0x6064, NULL, NULL, NULL, 0x0000 },
   /* Object 0x606C */
   {NULL,NULL, 0x606C, {DEFTYPE_INTEGER32 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x606C, aName0x606C, NULL, NULL, NULL, 0x0000 },
   /* Object 0x6077 */
   {NULL,NULL, 0x6077, {DEFTYPE_INTEGER16 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x6077, aName0x6077, NULL, NULL, NULL, 0x0000 },
   /* Object 0x607D */
   {NULL,NULL, 0x607D, {DEFTYPE_INTEGER32 , 2 | (OBJCODE_ARR << 8)}, asEntryDesc0x607D, aName0x607D, NULL, NULL, NULL, 0x0000 },
   /* Object 0x607A */
   {NULL,NULL, 0x607A, {DEFTYPE_INTEGER32 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x607A, aName0x607A, NULL, NULL, NULL, 0x0000 },
   /* Object 0x6081 */
    {NULL,NULL, 0x6081, {DEFTYPE_UNSIGNED32 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x6081, aName0x6081, NULL, NULL,
WriteMotionParameter, 0x0000 },
   /* Object 0x6083 */
    {NULL,NULL, 0x6083, {DEFTYPE_UNSIGNED32 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x6083, aName0x6083, NULL, NULL,
WriteMotionParameter, 0x0000 },
   /* Object 0x6084 */
    {NULL,NULL, 0x6084, {DEFTYPE_UNSIGNED32 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x6084, aName0x6084, NULL, NULL,
WriteMotionParameter, 0x0000 },
   /* Object 0x6085 */
    {NULL,NULL, 0x6085, {DEFTYPE_UNSIGNED32 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x6085, aName0x6085, NULL, NULL,
WriteMotionParameter, 0x0000 },
    {NULL,NULL, 0x2000, {DEFTYPE_UNSIGNED16, OBJCODE_VAR << 8}, &sEntryDesc0x2000, aName0x2000, NULL, NULL, WriteMotionParameter, 0},
    {NULL,NULL, 0x607F, {DEFTYPE_UNSIGNED32, OBJCODE_VAR << 8}, &sEntryDesc0x607F, aName0x607F, NULL, NULL, WriteMotionParameter, 0},
    {NULL,NULL, 0x60B0, {DEFTYPE_INTEGER32, OBJCODE_VAR << 8}, &sEntryDesc0x60B0, aName0x60B0, NULL, NULL, NULL, 0},
   /* Object 0x60C2 */
   {NULL,NULL, 0x60C2, {DEFTYPE_RECORD , 2 | (OBJCODE_REC << 8)}, asEntryDesc0x60C2, aName0x60C2, NULL, NULL, NULL, 0x0000 },
   /* Object 0x60FF */
   {NULL,NULL, 0x60FF, {DEFTYPE_INTEGER32 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x60FF, aName0x60FF, NULL, NULL, NULL, 0x0000 },
   /* Object 0x6502 */
   {NULL,NULL, 0x6502, {DEFTYPE_UNSIGNED32 , 0 | (OBJCODE_VAR << 8)}, &sEntryDesc0x6502, aName0x6502, NULL, NULL, NULL, 0x0000 },
   {NULL,NULL, 0xFFFF, {0, 0}, NULL, NULL, NULL, NULL, NULL, 0x000}}
#endif
;

PROTO void APPL_Application(void);

PROTO void   APPL_AckErrorInd(UINT16 stateTrans);
PROTO UINT16 APPL_StartMailboxHandler(void);
PROTO UINT16 APPL_StopMailboxHandler(void);
PROTO UINT16 APPL_StartInputHandler(UINT16 *pIntMask);
PROTO UINT16 APPL_StopInputHandler(void);
PROTO UINT16 APPL_StartOutputHandler(void);
PROTO UINT16 APPL_StopOutputHandler(void);

PROTO UINT16 APPL_GenerateMapping(UINT16 *pInputSize,UINT16 *pOutputSize);
PROTO void APPL_InputMapping(UINT16* pData);
PROTO void APPL_OutputMapping(UINT16* pData);

PROTO void CiA402_LocalError(UINT16 ErrorCode);
PROTO UINT16 CiA402_Init(void);
PROTO void CiA402_DeallocateAxis(void);
PROTO void CiA402_StateMachine(void);

#undef PROTO
/** @}*/
```

```
/*
* This source file is part of the EtherCAT Slave Stack Code licensed by Beckhoff Automation GmbH & Co KG, 33415 Verl, Germany.
* The corresponding license agreement applies. This hint shall not be removed.
* https://www.beckhoff.com/media/downloads/slave-stack-code/ethercat_ssc_license.pdf
*/

/**
 * \addtogroup CoE CAN Application Profile over EtherCAT
 * @{
 */

/**
\file coeappl.h
\author EthercatSSC@beckhoff.com

\version 5.13

<br>Changes to version V5.12:<br>
V5.13 COE6: add 0x10F0.2 (Backup parameter changed)<br>
<br>Changes to version V5.11:<br>
V5.12 COE4: add timestamp object (0x10F8) and update diagnosis handling<br>
V5.12 ECAT5: update Sync error counter/flag handling,check enum memory alignment depending on the processor,in case of a polled
timer disable ESC interrupts during DC_CheckWatchdog<br>
<br>Changes to version V5.01:<br>
V5.11 ECAT10: change PROTO handling to prevent compiler errors<br>
<br>Changes to version - :<br>
V5.01 : Start file change log
 */

/*-----------------------------------------------------------------------------------------
------
------    Includes
------
-----------------------------------------------------------------------------------------*/
#include "objdef.h"
#include "ecatappl.h"

#ifndef _COEAPPL_H_
#define _COEAPPL_H_

/*-----------------------------------------------------------------------------------------
------
------    Defines and Types
------
-----------------------------------------------------------------------------------------*/


/*-----------------------------------------------
-    0x1018
-----------------------------------------------*/
/**
* \brief 0x1018 (Identity) object structure
*/
typedef struct OBJ_STRUCT_PACKED_START {
    UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
    UINT32   u32VendorID; /**< \brief EtherCAT vendor ID */
```

```
    UINT32   u32Productcode; /**< \brief Product code*/
    UINT32   u32Revision; /**< \brief Revision number*/
    UINT32   u32Serialnumber; /**< \brief Serial number*/
} OBJ_STRUCT_PACKED_END
TOBJ1018;




/*-----------------------------------------------
-    0x1C00
-----------------------------------------------*/
/**
* \brief 0x1C00 (Sync manager type) object structure
*/
typedef struct OBJ_STRUCT_PACKED_START {
    UINT16   u16SubIndex0; /**< \brief SubIndex 0*/
    UINT16   aEntries[2]; /**< \brief Entry buffer (one for two SyncManagers)*/
} OBJ_STRUCT_PACKED_END
TOBJ1C00;

#define CHECK_DC_OVERRUN_IN_MS      4000 /**<\brief cycle time in ms to check the DC system time for overrun*/

#endif //_COEAPPL_H_

#if defined(_COEAPPL_) && (_COEAPPL_ == 1)
    #define PROTO
#else
    #define PROTO extern
#endif

/*---------------------------------------------------------------------------------------------
------
------    Global variables
------
---------------------------------------------------------------------------------------------*/

PROTO UINT64 u64Timestamp
#ifdef _COEAPPL_
= { 0 }
#endif
;

PROTO UINT32 u32LastDc32Value;
PROTO UINT32 u32CheckForDcOverrunCnt;
/*---------------------------------------------------------------------------------------------
------
------    Global functions
------
---------------------------------------------------------------------------------------------*/
PROTO void COE_UpdateSyncErrorStatus(void);

PROTO void COE_ObjInit(void);
PROTO void COE_Main(void);
```

```
PROTO UINT16 COE_ObjDictionaryInit(void);
PROTO UINT16 COE_AddObjectToDic(TOBJECT OBJMEM * pNewObjEntry);
PROTO void COE_RemoveDicEntry(UINT16 index);
PROTO void COE_ClearObjDictionary(void);
PROTO OBJCONST TOBJECT OBJMEM * COE_GetObjectDictionary(void);
PROTO void COE_SyncTimeStamp(void);

#undef PROTO
/** @}*/
/*
* This source file is part of the EtherCAT Slave Stack Code licensed by Beckhoff Automation GmbH & Co KG, 33415 Verl, Germany.
* The corresponding license agreement applies. This hint shall not be removed.
* https://www.beckhoff.com/media/downloads/slave-stack-code/ethercat_ssc_license.pdf
*/


/*-----------------------------------------------------------------------------------------
------
------  ecat_def.h
------  SSC version : 5.13
-----------------------------------------------------------------------------------------*/

#ifndef _ECATDEF_H_
#define _ECATDEF_H_

/*-----------------------------------------------------------------------------------------
------
------  Includes
------
-----------------------------------------------------------------------------------------*/
#include <stdlib.h>
#include <string.h>

/*-----------------------------------------------------------------------------------------
------
------  Slave Sample Code Configuration Defines
------
-----------------------------------------------------------------------------------------*/
#define SSC_VERSION_MAJOR 5
#define SSC_VERSION_MINOR 13


/**
EL9800_HW: Shall be set if the Slave code is executed on the PIC mounted on the EL9800 EtherCAT Evaluation Board.<br>
(if the MCI interface provided by EL9800 board should be used MCI_HW shall be set and this define shall be reset).<br>
This settings should also be enabled if the ESC is connected via a serial interface and no specific hardware access files are
avilable yet.<br>
NOTE: The PDI type needs also to be configured in the "ESC_CONFIG_DATA". */
#ifndef EL9800_HW
#define EL9800_HW                                   1 //This define was already evaluated by ET9300 Project Handler(V. 1.5.3.0)!
#endif

/**
MCI_HW: Shall be set if the MCI of the ESC is connected.<br>
This settings should also be enabled if the ESC is connected via a parallel interface and no specific hardware access files are
avilable yet.<br>
NOTE: The PDI type needs also to be configured in the "ESC_CONFIG_DATA". */
```

```c
/**
 * @brief 结束一个 USART1 TX DMA 段并继续发送下一段。
 */
void DMA1_Channel4_IRQHandler(void)
{
    if (DMA_GetIntStatus(DMA1_INT_TXC4, DMA1) != RESET)
    {
        /* 推进发送尾指针并清除忙标记，然后尝试发送队列剩余内容。 */
        DMA_ClrIntPendingBit(DMA1_INT_TXC4, DMA1);
        DMA_EnableChannel(UART_TX_DMA_CHANNEL, DISABLE);
        s_uart_tx_tail = (uint16_t)((s_uart_tx_tail + s_uart_tx_dma_length)
                                    % UART_BUFFER_SIZE);
        s_uart_tx_count += s_uart_tx_dma_length;
        s_uart_tx_dma_length = 0U;
        uart_start_tx_dma();
    }
}
#ifndef USER_SCI_INIT_H
#define USER_SCI_INIT_H

#include "mcuinit.h"

/* 串口 DMA 接收缓冲区大小（一帧最长字节数，按协议可调大） */
#define SCI_RX_BUF_SIZE     32
#define UART_BUFFER_SIZE      (256U)

void com_gpio_init(void);
void com_usart_init(void);
void SciReceive(void);
void SendFunction(u16 indata[]);
void CrcCal(u16 indata[]);
void RWFunction(u16 indata[]);

extern u16 CrcH;
extern u16 CrcL;
extern u8 RxSci_i;
extern u16 RxSciData[8];
extern u8 RxSciData_i;

/**
 * @brief Initialize the PA9/PA10 USART1 DMA command port.
 * @param[in] baudrate UART bit rate; zero selects 115200 bit/s.
 * @note Reception uses circular DMA and an idle-line interrupt.
 */
void uart_init(uint32_t baudrate);

/**
 * @brief Queue bytes for nonblocking USART1 DMA transmission.
 * @param[in] data Source bytes.
 * @param[in] length Number of bytes to queue.
 * @return Number of bytes accepted by the transmit queue.
 */
uint16_t uart_write(const uint8_t *data, uint16_t length);
```

```c
/**
 * @brief Read bytes received by USART1 from the foreground queue.
 * @param[out] data Destination buffer.
 * @param[in] capacity Maximum number of bytes to read.
 * @return Number of bytes copied to data.
 */
uint16_t uart_read(uint8_t *data, uint16_t capacity);

/**
 * @brief Start transmission of queued USART1 command responses.
 * @note Call repeatedly from the foreground loop; the operation is nonblocking.
 */
void uart_process(void);

#endif
#include "mcuinit.h"

/*
 * 定时器资源说明：
 *
 * TIM2：仅在脉冲/方向中断接口显式启用时使用 PA0、PA1；PA2 保持空闲。
 * TIM6：EtherCAT 协议栈本地时基，与 PWM/ADC 同步无关。
 * TIM8：四路 PWM 计数器和 ADC3 采样周期主定时器。
 *
 * PWM/ADC 硬件触发链：
 * TIM8 更新 TRGO -> ADC3 常规序列 -> DMA2_CH1 -> adc_value[]。
 */

/**
 * @brief 初始化 EtherCAT 协议栈使用的 TIM6 本地时基。
 * @param[in] period 原协议栈使用的定时周期倍数。
 */
void timer_init(uint8_t period)
{
    TIM_TimeBaseInitType timer_init;

#if ECAT_TIMER_INT
    NVIC_InitType nvic_init;
#endif

    RCC_EnableAPB1PeriphClk(RCC_APB1_PERIPH_TIM6,
                            ENABLE);

    TIM_DeInit(TIM6);
    timer_init.Period = (uint16_t)(period * 200U);
    timer_init.Prescaler = 41U;
    timer_init.ClkDiv = TIM_CLK_DIV1;
    timer_init.CntMode = TIM_CNT_MODE_UP;
    timer_init.RepetCnt = 0U;
    timer_init.CapCh1FromCompEn = false;
    timer_init.CapCh2FromCompEn = false;
    timer_init.CapCh3FromCompEn = false;
    timer_init.CapCh4FromCompEn = false;
    timer_init.CapEtrClrFromCompEn = false;
    timer_init.CapEtrSelFromTscEn = false;
```

```
    TIM_InitTimeBase(TIM6, &timer_init);
    TIM_ClearFlag(TIM6, TIM_FLAG_UPDATE);

#if ECAT_TIMER_INT
    TIM_ConfigInt(TIM6, TIM_INT_UPDATE, ENABLE);
#endif

    TIM_Enable(TIM6, ENABLE);

#if ECAT_TIMER_INT
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    nvic_init.NVIC_IRQChannel = TIM6_IRQn;
    /* 1 ms protocol timer runs below the ESC and SYNC event interrupts. */
    nvic_init.NVIC_IRQChannelPreemptionPriority = 2U;
    nvic_init.NVIC_IRQChannelSubPriority = 0U;
    nvic_init.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&nvic_init);
#endif
}

/**
 * @brief 初始化 PWM 和 ADC 共用的 TIM8 时基。
 *
 * 触发源：TIM8 更新事件输出的 TRGO 硬件信号。
 * ADC 触发模式：TIM8 更新 TRGO 直接启动 ADC3 常规序列。
 * 触发频率：16 kHz，即每 62.5 us 触发一组四路采样。
 *
 * TIM8 使用系统时钟、2 分频和中心对齐模式 1。计数器的完整周期为：
 *
 *     0 -> ARR -> 0
 *
 * 因此 PWM 频率计算式为：
 *
 *     PWM 频率 = 系统时钟 / ((PSC + 1) * 2 * (ARR + 1))
 *
 * 当前系统时钟为 144 MHz 时，PSC 为 1、ARR 为 2249，完整往返周期
 * 为 62.5 us，PWM 频率为 16 kHz。CCR 范围为 0～2250。
 *
 * 中心对齐模式在 ARR 顶部和 0 底部都会提出一次更新请求。RepetCnt 设为 1，
 * 让两个更新请求合成一次真正的更新事件，所以每个完整 PWM 周期只输出
 * 一次 TRGO。触发由 TIM8 硬件完成，不受 EtherCAT 中断执行时间影响。
 */
void timer_pwm_adc_init(void)
{
    TIM_TimeBaseInitType timer_init;
    RCC_ClocksType clocks;
    NVIC_InitType nvic_init;

    RCC_EnableAPB2PeriphClk(RCC_APB2_PERIPH_TIM8, ENABLE);
    RCC_ConfigTim18Clk(RCC_TIM18CLK_SRC_SYSCLK);
    RCC_GetClocksFreqValue(&clocks);

    TIM_DeInit(TIM8);
    TIM_InitTimBaseStruct(&timer_init);
    timer_init.Prescaler = 1U;
```

```
    timer_init.CntMode = TIM_CNT_MODE_CENTER_ALIGN1;
    timer_init.Period =
        (uint16_t)(clocks.SysclkFreq
                   / ((timer_init.Prescaler + 1U) * 2U * 16000U)
                   - 1U);
    timer_init.ClkDiv = TIM_CLK_DIV1;
    timer_init.RepetCnt = 1U;
    TIM_InitTimeBase(TIM8, &timer_init);

    TIM_ConfigArPreload(TIM8, ENABLE);
    TIM_SetCnt(TIM8, 0U);

    /*
     * 装载 PSC、ARR 和重复计数值时暂时不选择 UPDATE 作为 TRGO，避免初始化
     * 产生的软件更新事件提前启动一次 ADC 转换。
     */
    TIM_SelectOutputTrig(TIM8, TIM_TRGO_SRC_ENABLE);
    TIM_GenerateEvent(TIM8, TIM_EVT_SRC_UPDATE);
    TIM_ClearFlag(TIM8, TIM_FLAG_UPDATE);

    /* 每个完整 PWM 周期的更新事件直接输出一次硬件 TRGO。 */
    TIM_SelectOutputTrig(TIM8, TIM_TRGO_SRC_UPDATE);
    TIM_SelectMasterSlaveMode(TIM8,
                              TIM_MASTER_SLAVE_MODE_ENABLE);

    /* N32G455 的 TIM8 更新事件使用独立的 TIM8_UP 中断向量。 */
    TIM_ClearFlag(TIM8, TIM_FLAG_UPDATE);
    TIM_ConfigInt(TIM8, TIM_INT_UPDATE, ENABLE);

    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    nvic_init.NVIC_IRQChannel = TIM8_UP_IRQn;
    /* Match main.c: ESC IRQ (0) and SYNC0 (1) may preempt motor control. */
    nvic_init.NVIC_IRQChannelPreemptionPriority = 2U;
    nvic_init.NVIC_IRQChannelSubPriority = 0U;
    nvic_init.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&nvic_init);
}

/**
 * @brief 启动 PWM 和 ADC 共用的 TIM8。
 *
 * pwm_config()完成四路输出比较配置后调用本函数。TIM8 开始计数后：
 * 1. PC6～PC9 开始产生中心对齐 PWM；
 * 2. 每个完整 PWM 周期产生一次 TRGO；
 * 3. TIM8 更新 TRGO 直接启动 ADC3 常规序列；
 * 4. DMA 完成一轮 10 个结果后进入 DMA2_CH1 中断。
 */
void timer_pwm_adc_start(void)
{
    /* 从 0 开始，保证 PWM 周期和 ADC 触发相位在每次上电时一致。 */
    TIM_SetCnt(TIM8, 0U);
    TIM_ClearFlag(TIM8, TIM_FLAG_UPDATE);
    TIM_Enable(TIM8, ENABLE);
}
#ifndef USER_TIMER_H
```

```h
#define USER_TIMER_H

#include "N32G45XSYS.h"

/**
 * @brief 初始化 EtherCAT 协议栈使用的 TIM6 本地时基。
 * @param[in] period 原协议栈使用的定时周期倍数。
 * @note ECAT_TIMER_INT 非零时才打开 TIM6 更新中断。
 */
void timer_init(uint8_t period);

/**
 * @brief 初始化 PWM 和 ADC 共同使用的 TIM8 时基。
 *
 * TIM8 产生 16 kHz 中心对齐 PWM，并用更新 TRGO 直接触发 ADC3 常规序列。
 * 本函数只完成配置，不启动计数器。
 */
void timer_pwm_adc_init(void);

/**
 * @brief 启动 PWM 和 ADC 共同使用的 TIM8 计数器。
 * @note 调用前必须先完成 ADC 和 PWM 通道配置。
 */
void timer_pwm_adc_start(void);

#endif
#include "mcuinit.h"

#define LEGACY_I2C_EVENT_TIMEOUT (100000U)

static uint8_t s_legacy_i2c_error;
static uint8_t s_legacy_i2c_expect_address;

static uint8_t legacy_i2c_wait_event(uint32_t event)
{
    uint32_t timeout;

    timeout = LEGACY_I2C_EVENT_TIMEOUT;
    while (timeout > 0U)
    {
        if (I2C_CheckEvent(BSP_24C02_I2C, event) == SUCCESS)
        {
            return 0U;
        }

        if (I2C_GetFlag(BSP_24C02_I2C, I2C_FLAG_ACKFAIL) != RESET)
        {
            I2C_ClrFlag(BSP_24C02_I2C, I2C_FLAG_ACKFAIL);
            return 1U;
        }
        timeout--;
    }

    return 1U;
}
```

```
void delay_i2c(uint32_t nCount)
{
    volatile uint32_t delay;

    for (delay = 0U; delay < (nCount * 4U); delay++)
    {
        __NOP();
    }
}

void i2c_init(void)
{
    eeprom_init();
    s_legacy_i2c_error = 0U;
    s_legacy_i2c_expect_address = 0U;
}

void IIC_Start(void)
{
    I2C_ConfigAck(BSP_24C02_I2C, ENABLE);
    I2C_GenerateStart(BSP_24C02_I2C, ENABLE);
    s_legacy_i2c_error = legacy_i2c_wait_event(
        I2C_EVT_MASTER_MODE_FLAG);
    s_legacy_i2c_expect_address = 1U;
}

void IIC_Stop(void)
{
    I2C_GenerateStop(BSP_24C02_I2C, ENABLE);
    I2C_ConfigAck(BSP_24C02_I2C, ENABLE);
    s_legacy_i2c_expect_address = 0U;
}

uint8_t IIC_Wait_Ack(void)
{
    return s_legacy_i2c_error;
}

void IIC_Ack(void)
{
    I2C_ConfigAck(BSP_24C02_I2C, ENABLE);
}

void IIC_NAck(void)
{
    I2C_ConfigAck(BSP_24C02_I2C, DISABLE);
}

void IIC_Send_Byte(uint8_t txd)
{
    uint8_t address;
    uint16_t direction;

    if (s_legacy_i2c_error != 0U)
```

```
    {
        return;
    }

    if (s_legacy_i2c_expect_address != 0U)
    {
        direction = ((txd & 0x01U) != 0U)
                        ? I2C_DIRECTION_RECV
                        : I2C_DIRECTION_SEND;

        if ((txd == (BSP_24C02_DEVICE_ADDR_7BIT << 1U))
            || (txd == ((BSP_24C02_DEVICE_ADDR_7BIT << 1U) | 1U))
            || (txd == BSP_24C02_DEVICE_ADDR_7BIT)
            || (txd == (BSP_24C02_DEVICE_ADDR_7BIT | 1U)))
        {
            address = BSP_24C02_DEVICE_ADDR_7BIT;
        }
        else
        {
            address = (txd > 0x7FU) ? (uint8_t)(txd >> 1U) : txd;
        }

        I2C_SendAddr7bit(BSP_24C02_I2C, address, direction);
        s_legacy_i2c_error = legacy_i2c_wait_event(
            (direction == I2C_DIRECTION_RECV)
                ? I2C_EVT_MASTER_RXMODE_FLAG
                : I2C_EVT_MASTER_TXMODE_FLAG);
        s_legacy_i2c_expect_address = 0U;
    }
    else
    {
        I2C_SendData(BSP_24C02_I2C, txd);
        s_legacy_i2c_error = legacy_i2c_wait_event(
            I2C_EVT_MASTER_DATA_SENDED);
    }
}

uint8_t IIC_Read_Byte(unsigned char ack)
{
    if (ack != 0U)
    {
        IIC_Ack();
    }
    else
    {
        IIC_NAck();
    }

    s_legacy_i2c_error = legacy_i2c_wait_event(
        I2C_EVT_MASTER_DATA_RECVD_FLAG);
    if (s_legacy_i2c_error != 0U)
    {
        return 0U;
    }
```

```c
    return I2C_RecvData(BSP_24C02_I2C);
}

int32_t i2cWriteBuffer(uint8_t addr,
                       uint8_t reg,
                       uint8_t len,
                       uint8_t *data)
{
    if ((addr != (BSP_24C02_DEVICE_ADDR_7BIT << 1U))
        && (addr != BSP_24C02_DEVICE_ADDR_7BIT))
    {
        return -1;
    }
    return (eeprom_write(reg, data, len) == BSP_24C02_OK) ? 0 : -1;
}

int32_t i2cRead(uint8_t addr,
                uint8_t reg,
                uint8_t len,
                uint8_t *buf)
{
    if ((addr != (BSP_24C02_DEVICE_ADDR_7BIT << 1U))
        && (addr != BSP_24C02_DEVICE_ADDR_7BIT))
    {
        return -1;
    }
    return (eeprom_read(reg, buf, len) == BSP_24C02_OK) ? 0 : -1;
}
#ifndef USER_U_I2C_H
#define USER_U_I2C_H

#include "N32G45XSYS.h"

void i2c_init(void);
void IIC_Start(void);
void IIC_Stop(void);
uint8_t IIC_Wait_Ack(void);
void IIC_Ack(void);
void IIC_NAck(void);
void IIC_Send_Byte(uint8_t txd);
uint8_t IIC_Read_Byte(unsigned char ack);
int32_t i2cRead(uint8_t addr,
                uint8_t reg,
                uint8_t len,
                uint8_t *buf);
int32_t i2cWriteBuffer(uint8_t addr,
                       uint8_t reg,
                       uint8_t len,
                       uint8_t *data);
void delay_i2c(uint32_t nCount);

#define i2c_write(dev, reg, data, len) \
    (i2cWriteBuffer((dev), (reg), (len), (uint8_t *)(data)) == 0)
#define i2c_read(dev, reg, data, len) \
    (i2cRead((dev), (reg), (len), (uint8_t *)(data)) == 0)
```

```
#endif
#include "mcuinit.h"

//void UartIni(void)
//{
//    uart_init(4800U);
//}
#ifndef USER_UARTC_H
#define USER_UARTC_H

#include "N32G45XSYS.h"

void UartIni(void);
void SciReceive(void);

#endif
//#include "usmart.h"
//#include "usart.h"
//#include "sys.h"
//////////////////////////////////////////////////////////////////////////////////
////本程序只供学习使用，未经作者许可，不得用于其它任何用途
////ALIENTEK STM32 开发板
////正点原子@ALIENTEK
////技术论坛:www.openedv.com
////版本：V3.1
////版权所有，盗版必究。
////Copyright(C) 正点原子 2011-2021
////All rights reserved
////*********************************************************************************
////升级说明
////V1.4
////增加了对参数为 string 类型的函数的支持.适用范围大大提高.
////优化了内存占用,静态内存占用为 79 个字节@10 个参数.动态适应数字及字符串长度
////V2.0
////1,修改了 list 指令,打印函数的完整表达式.
////2,增加了 id 指令,打印每个函数的入口地址.
////3,修改了参数匹配,支持函数参数的调用(输入入口地址).
////4,增加了函数名长度宏定义.
////V2.1 20110707
////1,增加 dec,hex 两个指令,用于设置参数显示进制,及执行进制转换.
////注:当 dec,hex 不带参数的时候,即设定显示参数进制.当后跟参数的时候,即执行进制转换.
////如:"dec 0XFF" 则会将 0XFF 转为 255,由串口返回.
////如:"hex 100"     则会将 100 转为 0X64,由串口返回
////2,新增 usmart_get_cmdname 函数,用于获取指令名字.
////V2.2 20110726
////1,修正了 void 类型参数的参数统计错误.
////2,修改数据显示格式默认为 16 进制.
////V2.3 20110815
////1,去掉了函数名后必须跟"("的限制.
////2,修正了字符串参数中不能有"("的 bug.
////3,修改了函数默认显示参数格式的修改方式.
////V2.4 20110905
////1,修改了 usmart_get_cmdname 函数,增加最大参数长度限制.避免了输入错误参数时的死机现象.
////2,增加 USMART_ENTIM2_SCAN 宏定义,用于配置是否使用 TIM2 定时执行 scan 函数.
```

```
////V2.5 20110930
////1,修改 usmart_init 函数为 void usmart_init(u8 sysclk),可以根据系统频率自动设定扫描时间.(固定 100ms)
////2,去掉了 usmart_init 函数中的 uart_init 函数,串口初始化必须在外部初始化,方便用户自行管理.
////V2.6 20111009
////1,增加了 read_addr 和 write_addr 两个函数.可以利用这两个函数读写内部任意地址(必须是有效地址).更加方便调试.
////2,read_addr 和 write_addr 两个函数可以通过设置 USMART_USE_WRFUNS 为来使能和关闭.
////3,修改了 usmart_strcmp,使其规范化.
////V2.7 20111024
////1,修正了返回值 16 进制显示时不换行的 bug.
////2,增加了函数是否有返回值的判断,如果没有返回值,则不会显示.有返回值时才显示其返回值.
////V2.8 20111116
////1,修正了 list 等不带参数的指令发送后可能导致死机的 bug.
////V2.9 20120917
////1,修改了形如: void*xxx(void)类型函数不能识别的 bug。
////V3.0 20130425
////1,新增了字符串参数对转义符的支持。
////V3.1 20131120
////1,增加 runtime 系统指令,可以用于统计函数执行时间.
////用法:
////发送:runtime 1 ,则开启函数执行时间统计功能
////发送:runtime 0 ,则关闭函数执行时间统计功能
/////runtime 统计功能,必须设置:USMART_ENTIMX_SCAN 为 1,才可以使用!!
////////////////////////////////////////////////////////////////////////////////
////系统命令
//u8 *sys_cmd_tab[]=
//{
//  "?",
//  "help",
//  "list",
//  "id",
//  "hex",
//  "dec",
//  "runtime",
//};
////处理系统指令
////0,成功处理;其他,错误代码;
//u8 usmart_sys_cmd_exe(u8 *str)
//{
//  u8 i;
//  u8 sfname[MAX_FNAME_LEN];//存放本地函数名
//  u8 pnum;
//  u8 rval;
//  u32 res;
//  res=usmart_get_cmdname(str,sfname,&i,MAX_FNAME_LEN);//得到指令及指令长度
//  if(res)return USMART_FUNCERR;//错误的指令
//  str+=i;
//  for(i=0;i<sizeof(sys_cmd_tab)/4;i++)//支持的系统指令
//  {
//      if(usmart_strcmp(sfname,sys_cmd_tab[i])==0)break;
//  }
//  switch(i)
//  {
//      case 0:
//      case 1://帮助指令
//          printf("\r\n");
```

```
//#if USMART_USE_HELP
//          printf("------------------------USMART V3.1------------------------ \r\n");
//          printf("    USMART 是由 ALIENTEK 开发的一个灵巧的串口调试互交组件,通过 \r\n");
//          printf("它,你可以通过串口助手调用程序里面的任何函数,并执行.因此,你可\r\n");
//          printf("以随意更改函数的输入参数(支持数字(10/16 进制)、字符串、函数入\r\n");
//          printf("口地址等作为参数),单个函数最多支持 10 个输入参数,并支持函数返 \r\n");
//          printf("回值显示.新增参数显示进制设置功能,新增进制转换功能.\r\n");
//          printf("技术支持:www.openedv.com\r\n");
//          printf("USMART 有 7 个系统命令:\r\n");
//          printf("?:      获取帮助信息\r\n");
//          printf("help:   获取帮助信息\r\n");
//          printf("list:   可用的函数列表\r\n\n");
//          printf("id:     可用函数的 ID 列表\r\n\n");
//          printf("hex:    参数 16 进制显示,后跟空格+数字即执行进制转换\r\n\n");
//          printf("dec:    参数 10 进制显示,后跟空格+数字即执行进制转换\r\n\n");
//          printf("runtime:1,开启函数运行计时;0,关闭函数运行计时;\r\n\n");
//          printf("请按照程序编写格式输入函数名及参数并以回车键结束.\r\n");
//          printf("------------------------ALIENTEK------------------------ \r\n");
//#else
//          printf("指令失效\r\n");
//#endif
//          break;
//      case 2://查询指令
//          printf("\r\n");
//          printf("-------------------------函数清单-------------------------- \r\n");
//          for(i=0;i<usmart_dev.fnum;i++)printf("%s\r\n",usmart_dev.funs[i].name);
//          printf("\r\n");
//          break;
//      case 3://查询 ID
//          printf("\r\n");
//          printf("-------------------------函数 ID --------------------------- \r\n");
//          for(i=0;i<usmart_dev.fnum;i++)
//          {
//              usmart_get_fname((u8*)usmart_dev.funs[i].name,sfname,&pnum,&rval);//得到本地函数名
//              printf("%s id is:\r\n0X%08X\r\n",sfname,usmart_dev.funs[i].func); //显示 ID
//          }
//          printf("\r\n");
//          break;
//      case 4://hex 指令
//          printf("\r\n");
//          usmart_get_aparm(str,sfname,&i);
//          if(i==0)//参数正常
//          {
//              i=usmart_str2num(sfname,&res);      //记录该参数
//              if(i==0)                            //进制转换功能
//              {
//                  printf("HEX:0X%X\r\n",res);     //转为 16 进制
//              }else if(i!=4)return USMART_PARMERR;//参数错误.
//              else                                //参数显示设定功能
//              {
//                  printf("16 进制参数显示!\r\n");
//                  usmart_dev.sptype=SP_TYPE_HEX;
//              }

//          }else return USMART_PARMERR;            //参数错误.
```

```
//          printf("\r\n");
//          break;
//      case 5://dec 指令
//          printf("\r\n");
//          usmart_get_aparm(str,sfname,&i);
//          if(i==0)//参数正常
//          {
//              i=usmart_str2num(sfname,&res);      //记录该参数
//              if(i==0)                            //进制转换功能
//              {
//                  printf("DEC:%lu\r\n",res);      //转为 10 进制
//              }else if(i!=4)return USMART_PARMERR;//参数错误.
//              else                                //参数显示设定功能
//              {
//                  printf("10 进制参数显示!\r\n");
//                  usmart_dev.sptype=SP_TYPE_DEC;
//              }

//          }else return USMART_PARMERR;            //参数错误.
//          printf("\r\n");
//          break;
//      case 6://runtime 指令,设置是否显示函数执行时间
//          printf("\r\n");
//          usmart_get_aparm(str,sfname,&i);
//          if(i==0)//参数正常
//          {
//              i=usmart_str2num(sfname,&res);          //记录该参数
//              if(i==0)                                //读取指定地址数据功能
//              {
//                  if(USMART_ENTIMX_SCAN==0)printf("\r\nError! \r\nTo EN RunTime function,Please set USMART_ENTIMX_SCAN = 1
first!\r\n");//报错
//                  else
//                  {
//                      usmart_dev.runtimeflag=res;
//                      if(usmart_dev.runtimeflag)printf("Run Time Calculation ON\r\n");
//                      else printf("Run Time Calculation OFF\r\n");
//                  }
//              }else return USMART_PARMERR;            //未带参数,或者参数错误
//          }else return USMART_PARMERR;                //参数错误.
//          printf("\r\n");
//          break;
//      default://非法指令
//          return USMART_FUNCERR;
//  }
//  return 0;
//}
////////////////////////////////////////////////////////////////////////////////////////
////移植注意:本例是以 stm32 为例,如果要移植到其他 mcu,请做相应修改.
////usmart_reset_runtime,清除函数运行时间,连同定时器的计数寄存器以及标志位一起清零.并设置重装载值为最大,以最大限度的延长计时时间.
////usmart_get_runtime,获取函数运行时间,通过读取 CNT 值获取,由于 usmart 是通过中断调用的函数,所以定时器中断不再有效,此时最大限度
////只能统计 2 次 CNT 的值,也就是清零后+溢出一次,当溢出超过 2 次,没法处理,所以最大延时,控制在:2*计数器 CNT*0.1ms.对 STM32 来说,是:13.1s 左右
////其他的:TIM4_IRQHandler 和 Timer4_Init,需要根据 MCU 特点自行修改.确保计数器计数频率为:10Khz 即可.另外,定时器不要开启自动重装载功能!!

//#if USMART_ENTIMX_SCAN==1
////复位 runtime
////需要根据所移植到的 MCU 的定时器参数进行修改
```

```
//void usmart_reset_runtime(void)
//{
//
//  TIM_ClearFlag(TIM4,TIM_FLAG_UPDATE);//清除中断标志位
//  TIM_SetAutoReload(TIM4,0XFFFF);//将重装载值设置到最大
//  TIM_SetCnt(TIM4,0);     //清空定时器的 CNT
//  usmart_dev.runtime=0;
//}
////获得 runtime 时间
////返回值:执行时间,单位:0.1ms,最大延时时间为定时器 CNT 值的 2 倍*0.1ms
////需要根据所移植到的 MCU 的定时器参数进行修改
//u32 usmart_get_runtime(void)
//{
//  if(TIM_GetFlagStatus(TIM4,TIM_FLAG_UPDATE)==SET)//在运行期间,产生了定时器溢出
//  {
//      usmart_dev.runtime+=0XFFFF;
//  }
//  usmart_dev.runtime+=TIM_GetCnt(TIM4);
//  return usmart_dev.runtime;      //返回计数值
//}
////下面这两个函数,非 USMART 函数,放到这里,仅仅方便移植.
////定时器 4 中断服务程序
//void TIM4_IRQHandler(void)
//{
//  if(TIM_GetIntStatus(TIM4,TIM_INT_UPDATE)==SET)//溢出中断
//  {
//      usmart_dev.scan();  //执行 usmart 扫描
//      TIM_SetCnt(TIM4,0);     //清空定时器的 CNT
//      TIM_SetAutoReload(TIM4,100);//恢复原来的设置
//  }
//  TIM_ClrIntPendingBit(TIM4,TIM_INT_UPDATE);  //清除中断标志位
//}
////使能定时器 4,使能中断.
//void Timer4_Init(u16 arr,u16 psc)
//{
//  NVIC_InitType   NVIC_InitStructure;
//  TIM_TimeBaseInitType TIM_TimeBaseInitStructure;

//  RCC_EnableAPB1PeriphClk(RCC_APB1_PERIPH_TIM4,ENABLE);  ///使能 TIM4 时钟

//  TIM_TimeBaseInitStructure.Prescaler=psc;  //定时器分频
//  TIM_TimeBaseInitStructure.CntMode=TIM_CNT_MODE_UP; //向上计数模式
//  TIM_TimeBaseInitStructure.Period=arr;   //自动重装载值
//  TIM_TimeBaseInitStructure.ClkDiv=TIM_CLK_DIV1;
//
//  TIM_InitTimeBase(TIM4,&TIM_TimeBaseInitStructure);//初始化定时器 4
//
//  TIM_ConfigInt(TIM4,TIM_INT_UPDATE,ENABLE); //允许定时器 4 更新中断
//  TIM_Enable(TIM4,ENABLE); //使能定时器 4
//
//  NVIC_InitStructure.NVIC_IRQChannel = TIM4_IRQn;//外部中断 4
//  NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 3;//抢占优先级 3
//  NVIC_InitStructure.NVIC_IRQChannelSubPriority = 3;//子优先级 3
//  NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;//使能外部中断通道
//  NVIC_Init(&NVIC_InitStructure);//配置 NVIC
```

```
//
//}
//#endif
////////////////////////////////////////////////////////////////////////////////
////初始化串口控制器
////sysclk:系统时钟（Mhz）
//void usmart_init(u8 sysclk)
//{
//#if USMART_ENTIMX_SCAN==1
//  Timer4_Init(1000,(u32)sysclk*100-1);//分频,时钟为 10K ,100ms 中断一次,注意,计数频率必须为 10Khz,以和 runtime 单位(0.1ms)同步.
//#endif
//  usmart_dev.sptype=1;    //十六进制显示参数
//}
////从 str 中获取函数名,id,及参数信息
////*str:字符串指针.
////返回值:0,识别成功;其他,错误代码.
//u8 usmart_cmd_rec(u8*str)
//{
//  u8 sta,i,rval;//状态
//  u8 rpnum,spnum;
//  u8 rfname[MAX_FNAME_LEN];//暂存空间,用于存放接收到的函数名
//  u8 sfname[MAX_FNAME_LEN];//存放本地函数名
//  sta=usmart_get_fname(str,rfname,&rpnum,&rval);//得到接收到的数据的函数名及参数个数
//  if(sta)return sta;//错误
//  for(i=0;i<usmart_dev.fnum;i++)
//  {
//      sta=usmart_get_fname((u8*)usmart_dev.funs[i].name,sfname,&spnum,&rval);//得到本地函数名及参数个数
//      if(sta)return sta;//本地解析有误
//      if(usmart_strcmp(sfname,rfname)==0)//相等
//      {
//          if(spnum>rpnum)return USMART_PARMERR;//参数错误(输入参数比源函数参数少)
//          usmart_dev.id=i;//记录函数 ID.
//          break;//跳出.
//      }
//  }
//  if(i==usmart_dev.fnum)return USMART_NOFUNCFIND; //未找到匹配的函数
//  sta=usmart_get_fparam(str,&i);                  //得到函数参数个数
//  if(sta)return sta;                              //返回错误
//  usmart_dev.pnum=i;                              //参数个数记录
//    return USMART_OK;
//}
////usamrt 执行函数
////该函数用于最终执行从串口收到的有效函数.
////最多支持 10 个参数的函数,更多的参数支持也很容易实现.不过用的很少.一般 5 个左右的参数的函数已经很少见了.
////该函数会在串口打印执行情况.以:"函数名(参数 1, 参数 2...参数 N)=返回值".的形式打印.
////当所执行的函数没有返回值的时候,所打印的返回值是一个无意义的数据.
//void usmart_exe(void)
//{
//  u8 id,i;
//  u32 res;
//  u32 temp[MAX_PARM];//参数转换,使之支持了字符串
//  u8 sfname[MAX_FNAME_LEN];//存放本地函数名
//  u8 pnum,rval;
//  id=usmart_dev.id;
//  if(id>=usmart_dev.fnum)return;//不执行.
```

```
//  usmart_get_fname((u8*)usmart_dev.funs[id].name,sfname,&pnum,&rval);//得到本地函数名,及参数个数
//  printf("\r\n%s(",sfname);//输出正要执行的函数名
//  for(i=0;i<pnum;i++)//输出参数
//  {
//      if(usmart_dev.parmtype&(1<<i))//参数是字符串
//      {
//          printf("%c",'"');
//          printf("%s",usmart_dev.parm+usmart_get_parmpos(i));
//          printf("%c",'"');
//          temp[i]=(u32)&(usmart_dev.parm[usmart_get_parmpos(i)]);
//      }else                           //参数是数字
//      {
//          temp[i]=*(u32*)(usmart_dev.parm+usmart_get_parmpos(i));
//          if(usmart_dev.sptype==SP_TYPE_DEC)printf("%lu",temp[i]);//10 进制参数显示
//          else printf("0X%X",temp[i]);//16 进制参数显示
//      }
//      if(i!=pnum-1)printf(",");
//  }
//  printf(")");
//  usmart_reset_runtime(); //计时器清零,开始计时
//  switch(usmart_dev.pnum)
//  {
//      case 0://无参数(void 类型)
//          res=(*(u32(*)())usmart_dev.funs[id].func)();
//          break;
//      case 1://有 1 个参数
//          res=(*(u32(*)())usmart_dev.funs[id].func)(temp[0]);
//          break;
//      case 2://有 2 个参数
//          res=(*(u32(*)())usmart_dev.funs[id].func)(temp[0],temp[1]);
//          break;
//      case 3://有 3 个参数
//          res=(*(u32(*)())usmart_dev.funs[id].func)(temp[0],temp[1],temp[2]);
//          break;
//      case 4://有 4 个参数
//          res=(*(u32(*)())usmart_dev.funs[id].func)(temp[0],temp[1],temp[2],temp[3]);
//          break;
//      case 5://有 5 个参数
//          res=(*(u32(*)())usmart_dev.funs[id].func)(temp[0],temp[1],temp[2],temp[3],temp[4]);
//          break;
//      case 6://有 6 个参数
//          res=(*(u32(*)())usmart_dev.funs[id].func)(temp[0],temp[1],temp[2],temp[3],temp[4],\
//          temp[5]);
//          break;
//      case 7://有 7 个参数
//          res=(*(u32(*)())usmart_dev.funs[id].func)(temp[0],temp[1],temp[2],temp[3],temp[4],\
//          temp[5],temp[6]);
//          break;
//      case 8://有 8 个参数
//          res=(*(u32(*)())usmart_dev.funs[id].func)(temp[0],temp[1],temp[2],temp[3],temp[4],\
//          temp[5],temp[6],temp[7]);
//          break;
//      case 9://有 9 个参数
//          res=(*(u32(*)())usmart_dev.funs[id].func)(temp[0],temp[1],temp[2],temp[3],temp[4],\
//          temp[5],temp[6],temp[7],temp[8]);
```

```
//          break;
//      case 10://有 10 个参数
//          res=(*(u32(*)())usmart_dev.funs[id].func)(temp[0],temp[1],temp[2],temp[3],temp[4],\
//          temp[5],temp[6],temp[7],temp[8],temp[9]);
//          break;
//  }
//  usmart_get_runtime();//获取函数执行时间
//  if(rval==1)//需要返回值.
//  {
//      if(usmart_dev.sptype==SP_TYPE_DEC)printf("=%lu;\r\n",res);//输出执行结果(10 进制参数显示)
//      else printf("=0X%X;\r\n",res);//输出执行结果(16 进制参数显示)
//  }else printf(";\r\n");      //不需要返回值,直接输出结束
//  if(usmart_dev.runtimeflag)  //需要显示函数执行时间
//  {
//      printf("Function Run Time:%d.%1dms\r\n",usmart_dev.runtime/10,usmart_dev.runtime%10);//打印函数执行时间
//  }
//}
////usmart 扫描函数
////通过调用该函数,实现 usmart 的各个控制.该函数需要每隔一定时间被调用一次
////以及时执行从串口发过来的各个函数.
////本函数可以在中断里面调用,从而实现自动管理.
////如果非 ALIENTEK 用户,则 USART_RX_STA 和 USART_RX_BUF[]需要用户自己实现
//void usmart_scan(void)
//{
//  u8 sta,len;
//  if(USART_RX_STA&0x8000)//串口接收完成?
//  {
//    printf("\r\n 串口接收完成 ");
//      len=USART_RX_STA&0x3fff;    //得到此次接收到的数据长度
//      USART_RX_BUF[len]='\0'; //在末尾加入结束符.
//      sta=usmart_dev.cmd_rec(USART_RX_BUF);//得到函数各个信息
//      if(sta==0)usmart_dev.exe(); //执行函数
//      else
//      {
//          len=usmart_sys_cmd_exe(USART_RX_BUF);
//          if(len!=USMART_FUNCERR)sta=len;
//          if(sta)
//          {
//              switch(sta)
//              {
//                  case USMART_FUNCERR:
//                      printf("函数错误!\r\n");
//                      break;
//                  case USMART_PARMERR:
//                      printf("参数错误!\r\n");
//                      break;
//                  case USMART_PARMOVER:
//                      printf("参数太多!\r\n");
//                      break;
//                  case USMART_NOFUNCFIND:
//                      printf("未找到匹配的函数!\r\n");
//                      break;
//              }
//          }
//      }
```

```
//      USART_RX_STA=0;//状态寄存器清空
//  }
//}

//#if USMART_USE_WRFUNS==1  //如果使能了读写操作
////读取指定地址的值
//u32 read_addr(u32 addr)
//{
//  return *(u32*)addr;//
//}
////在指定地址写入指定的值
//void write_addr(u32 addr,u32 val)
//{
//  *(u32*)addr=val;
//}
//#endif
```

```
#ifndef __USMART_H
#define __USMART_H
#include "usmart_str.h"
////////////////////////////////////////////////////////////////////////////////
//本程序只供学习使用，未经作者许可，不得用于其它任何用途
//ALIENTEK STM32开发板
//正点原子@ALIENTEK
//技术论坛:www.openedv.com
//版本：V3.1
//版权所有，盗版必究。
//Copyright(C) 正点原子 2011-2021
//All rights reserved
//********************************************************************************
//升级说明
//V1.4
//增加了对参数为string类型的函数的支持.适用范围大大提高.
//优化了内存占用,静态内存占用为79个字节@10个参数.动态适应数字及字符串长度
//V2.0
//1,修改了list指令,打印函数的完整表达式.
//2,增加了id指令,打印每个函数的入口地址.
//3,修改了参数匹配,支持函数参数的调用(输入入口地址).
//4,增加了函数名长度宏定义.
//V2.1 20110707
//1,增加dec,hex两个指令,用于设置参数显示进制,及执行进制转换.
//注:当dec,hex不带参数的时候,即设定显示参数进制.当后跟参数的时候,即执行进制转换.
//如:"dec 0XFF" 则会将0XFF转为255,由串口返回.
```

```
//如:"hex 100"    则会将 100 转为 0X64,由串口返回
//2,新增 usmart_get_cmdname 函数,用于获取指令名字.
//V2.2 20110726
//1,修正了 void 类型参数的参数统计错误.
//2,修改数据显示格式默认为 16 进制.
//V2.3 20110815
//1,去掉了函数名后必须跟"("的限制.
//2,修正了字符串参数中不能有"("的 bug.
//3,修改了函数默认显示参数格式的修改方式.
//V2.4 20110905
//1,修改了 usmart_get_cmdname 函数,增加最大参数长度限制.避免了输入错误参数时的死机现象.
//2,增加 USMART_ENTIM2_SCAN 宏定义,用于配置是否使用 TIM2 定时执行 scan 函数.
//V2.5 20110930
//1,修改 usmart_init 函数为 void usmart_init(u8 sysclk),可以根据系统频率自动设定扫描时间.(固定 100ms)
//2,去掉了 usmart_init 函数中的 uart_init 函数,串口初始化必须在外部初始化,方便用户自行管理.
//V2.6 20111009
//1,增加了 read_addr 和 write_addr 两个函数.可以利用这两个函数读写内部任意地址(必须是有效地址).更加方便调试.
//2,read_addr 和 write_addr 两个函数可以通过设置 USMART_USE_WRFUNS 为来使能和关闭.
//3,修改了 usmart_strcmp,使其规范化.
//V2.7 20111024
//1,修正了返回值 16 进制显示时不换行的 bug.
//2,增加了函数是否有返回值的判断,如果没有返回值,则不会显示.有返回值时才显示其返回值.
//V2.8 20111116
//1,修正了 list 等不带参数的指令发送后可能导致死机的 bug.
//V2.9 20120917
//1,修改了形如：void*xxx(void)类型函数不能识别的 bug。
//V3.0 20130425
//1,新增了字符串参数对转义符的支持。
//V3.1 20131120
//1,增加 runtime 系统指令,可以用于统计函数执行时间.
//用法:
//发送:runtime 1 ,则开启函数执行时间统计功能
//发送:runtime 0 ,则关闭函数执行时间统计功能
///runtime 统计功能,必须设置:USMART_ENTIMX_SCAN 为 1,才可以使用!!
////////////////////////////////////////////////////////////////////////////////
//USMART 资源占用情况@MDK 3.80A@2.0 版本：
//FLASH:4K~K 字节(通过 USMART_USE_HELP 和 USMART_USE_WRFUNS 设置)
//SRAM:72 字节(最少的情况下)
//SRAM 计算公式:   SRAM=PARM_LEN+72-4  其中 PARM_LEN 必须大于等于 4.
//应该保证堆栈不小于 100 个字节.
/////////////////////////////////////////////用户配置参数/////////////////////////////////////////////////////
#define MAX_FNAME_LEN       30  //函数名最大长度，应该设置为不小于最长函数名的长度。
#define MAX_PARM            10  //最大为 10 个参数 ,修改此参数,必须修改 usmart_exe 与之对应.
#define PARM_LEN            200 //所有参数之和的长度不超过 PARM_LEN 个字节,注意串口接收部分要与之对应(不小于 PARM_LEN)


#define USMART_ENTIMX_SCAN  1   //使用 TIM 的定时中断来扫描 SCAN 函数,如果设置为 0,需要自己实现隔一段时间扫描一次 scan 函数.
                                //注意:如果要用 runtime 统计功能,必须设置 USMART_ENTIMX_SCAN 为 1!!!!

#define USMART_USE_HELP     1   //使用帮助，该值设为 0，可以节省近 700 个字节，但是将导致无法显示帮助信息。
#define USMART_USE_WRFUNS   1   //使用读写函数,使能这里,可以读取任何地址的值,还可以写寄存器的值.
/////////////////////////////////////////////END/////////////////////////////////////////////////////////////

#define USMART_OK           0  //无错误
#define USMART_FUNCERR      1  //函数错误
```

```
#define USMART_PARMERR      2  //参数错误
#define USMART_PARMOVER     3  //参数溢出
#define USMART_NOFUNCFIND   4  //未找到匹配函数

#define SP_TYPE_DEC         0  //10 进制参数显示
#define SP_TYPE_HEX         1  //16 进制参数显示


 //函数名列表
struct _m_usmart_nametab
{
    void* func;         //函数指针
    const u8* name;     //函数名(查找串)
};
//usmart 控制管理器
struct _m_usmart_dev
{
    struct _m_usmart_nametab *funs; //函数名指针

    void (*init)(u8);               //初始化
    u8 (*cmd_rec)(u8*str);          //识别函数名及参数
    void (*exe)(void);              //执行
    void (*scan)(void);             //扫描
    u8 fnum;                        //函数数量
    u8 pnum;                        //参数数量
    u8 id;                          //函数 id
    u8 sptype;                      //参数显示类型(非字符串参数):0,10 进制;1,16 进制;
    u16 parmtype;                   //参数的类型
    u8  plentbl[MAX_PARM];          //每个参数的长度暂存表
    u8  parm[PARM_LEN];             //函数的参数
    u8 runtimeflag;                 //0,不统计函数执行时间;1,统计函数执行时间,注意:此功能必须在 USMART_ENTIMX_SCAN 使能的时候,才有用
    u32 runtime;                    //运行时间,单位:0.1ms,最大延时时间为定时器 CNT 值的 2 倍*0.1ms
};
extern struct _m_usmart_nametab usmart_nametab[];   //在 usmart_config.c 里面定义
extern struct _m_usmart_dev usmart_dev;             //在 usmart_config.c 里面定义


void usmart_init(u8 sysclk);//初始化
u8 usmart_cmd_rec(u8*str);  //识别
void usmart_exe(void);      //执行
void usmart_scan(void);     //扫描
u32 read_addr(u32 addr);    //读取指定地址的值
void write_addr(u32 addr,u32 val);//在指定地址写入指定的值
u32 usmart_get_runtime(void);   //获取运行时间
void usmart_reset_runtime(void);//复位运行时间

#endif
```

```
//#include "usmart.h"
//#include "usmart_str.h"
////////////////////////////用户配置区///////////////////////////////////////////////
////这下面要包含所用到的函数所申明的头文件(用户自己添加)
//#include "delay.h"
//#include "sys.h"
//#include "el9800hw.h"
//#include "ecatappl.h"
//#include "SPI1.h"
//#include "ecat_def.h"

//extern void led_set(u8 sta);
//extern void test_fun(void(*ledset)(u8),u8 sta);
////函数名列表初始化(用户自己添加)
////用户直接在这里输入要执行的函数名及其查找串
//struct _m_usmart_nametab usmart_nametab[]=
//{
//#if USMART_USE_WRFUNS==1  //如果使能了读写操作
//  (void*)read_addr,"u32 read_addr(u32 addr)",
//  (void*)write_addr,"void write_addr(u32 addr,u32 val)",
//#endif
//  (void*)delay_ms,"void delay_ms(u16 nms)",
//  (void*)delay_us,"void delay_us(u32 nus)",
//  (void*)SPIReadDRegister,"void SPIReadDRegister(uint8_t *ReadBuffer, uint16_t Address, uint16_t Count)",
//  (void*)SPIWriteRegister,"void SPIWriteRegister( uint8_t *WriteBuffer, uint16_t Address, uint16_t Count)",
//  (void*)PDIReadLAN9252DirectReg,"PDIReadLAN9252DirectReg( UINT16 Address)",
//  (void*)PDIWriteLAN9252DirectReg,"void PDIWriteLAN9252DirectReg( UINT32 Val, UINT16 Address)",
//  (void*)ET1100_EscWriteDWord,"void ET1100_EscWriteDWord(UINT16 Address,UINT32 Value)",
//  (void*)ET1100_EscReadDWord,"void ET1100_EscReadDWord(UINT16 Address)",
//  (void*)ET1100_EscReadWord,"void ET1100_EscReadWord(UINT16 Address)",
//  (void*)ET1100_EscWriteWord,"void ET1100_EscWriteWord(UINT16 Address,UINT16 Value)",
////    (void*)ET1100_EscWrite_UserRAM,"void ET1100_EscWrite_UserRAM(void)",
////    (void*)ET1100_EscRead_AllRegister,"void ET1100_EscRead_AllRegister(void)",
```

```
////    (void*)test_fun,"void test_fun(void(*ledset)(u8),u8 sta)",
//};
///////////////////////////////////END/////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////////////////
////函数控制管理器初始化
////得到各个受控函数的名字
////得到函数总数量
//struct _m_usmart_dev usmart_dev=
//{
//  usmart_nametab,
//  usmart_init,
//  usmart_cmd_rec,
//  usmart_exe,
//  usmart_scan,
//  sizeof(usmart_nametab)/sizeof(struct _m_usmart_nametab),//函数数量
//  0,      //参数数量
//  0,      //函数 ID
//  1,      //参数显示类型,0,10 进制;1,16 进制
//  0,      //参数类型.bitx:,0,数字;1,字符串
//  0,      //每个参数的长度暂存表,需要 MAX_PARM 个 0 初始化
//  0,      //函数的参数,需要 PARM_LEN 个 0 初始化
//};
```

```
//#include "usmart_str.h"
//#include "usmart.h"
////////////////////////////////////////////////////////////////////////////////
////本程序只供学习使用，未经作者许可，不得用于其它任何用途
////ALIENTEK STM32 开发板
////正点原子@ALIENTEK
////技术论坛:www.openedv.com
////版本：V3.1
////版权所有，盗版必究。
////Copyright(C) 正点原子 2011-2021
////All rights reserved
////********************************************************************************
////升级说明
////V1.4
```

```
////增加了对参数为 string 类型的函数的支持.适用范围大大提高.
////优化了内存占用,静态内存占用为 79 个字节@10 个参数.动态适应数字及字符串长度
////V2.0
////1,修改了 list 指令,打印函数的完整表达式.
////2,增加了 id 指令,打印每个函数的入口地址.
////3,修改了参数匹配,支持函数参数的调用(输入入口地址).
////4,增加了函数名长度宏定义.
////V2.1 20110707
////1,增加 dec,hex 两个指令,用于设置参数显示进制,及执行进制转换.
////注:当 dec,hex 不带参数的时候,即设定显示参数进制.当后跟参数的时候,即执行进制转换.
////如:"dec 0XFF" 则会将 0XFF 转为 255,由串口返回.
////如:"hex 100"     则会将 100 转为 0X64,由串口返回
////2,新增 usmart_get_cmdname 函数,用于获取指令名字.
////V2.2 20110726
////1,修正了 void 类型参数的参数统计错误.
////2,修改数据显示格式默认为 16 进制.
////V2.3 20110815
////1,去掉了函数名后必须跟"("的限制.
////2,修正了字符串参数中不能有"("的 bug.
////3,修改了函数默认显示参数格式的修改方式.
////V2.4 20110905
////1,修改了 usmart_get_cmdname 函数,增加最大参数长度限制.避免了输入错误参数时的死机现象.
////2,增加 USMART_ENTIM2_SCAN 宏定义,用于配置是否使用 TIM2 定时执行 scan 函数.
////V2.5 20110930
////1,修改 usmart_init 函数为 void usmart_init(u8 sysclk),可以根据系统频率自动设定扫描时间.(固定 100ms)
////2,去掉了 usmart_init 函数中的 uart_init 函数,串口初始化必须在外部初始化,方便用户自行管理.
////V2.6 20111009
////1,增加了 read_addr 和 write_addr 两个函数.可以利用这两个函数读写内部任意地址(必须是有效地址).更加方便调试.
////2,read_addr 和 write_addr 两个函数可以通过设置 USMART_USE_WRFUNS 为来使能和关闭.
////3,修改了 usmart_strcmp,使其规范化.
////V2.7 20111024
////1,修正了返回值 16 进制显示时不换行的 bug.
////2,增加了函数是否有返回值的判断,如果没有返回值,则不会显示.有返回值时才显示其返回值.
////V2.8 20111116
////1,修正了 list 等不带参数的指令发送后可能导致死机的 bug.
////V2.9 20120917
////1,修改了形如: void*xxx(void)类型函数不能识别的 bug。
////V3.0 20130425
////1,新增了字符串参数对转义符的支持。
////V3.1 20131120
////1,增加 runtime 系统指令,可以用于统计函数执行时间.
////用法:
////发送:runtime 1 ,则开启函数执行时间统计功能
////发送:runtime 0 ,则关闭函数执行时间统计功能
/////runtime 统计功能,必须设置:USMART_ENTIMX_SCAN 为 1,才可以使用!!
////////////////////////////////////////////////////////////////////////////////
//
////对比字符串 str1 和 str2
////*str1:字符串 1 指针
////*str2:字符串 2 指针
////返回值:0, 相等;1, 不相等;
//u8 usmart_strcmp(u8 *str1,u8 *str2)
//{
//  while(1)
//  {
```

```
//      if(*str1!=*str2)return 1;//不相等
//      if(*str1=='\0')break;//对比完成了.
//      str1++;
//      str2++;
//  }
//  return 0;//两个字符串相等
//}
////把 str1 的内容 copy 到 str2
////*str1:字符串 1 指针
////*str2:字符串 2 指针
//void usmart_strcopy(u8*str1,u8 *str2)
//{
//  while(1)
//  {
//      *str2=*str1;    //拷贝
//      if(*str1=='\0')break;//拷贝完成了.
//      str1++;
//      str2++;
//  }
//}
////得到字符串的长度(字节)
////*str:字符串指针
////返回值:字符串的长度
//u8 usmart_strlen(u8*str)
//{
//  u8 len=0;
//  while(1)
//  {
//      if(*str=='\0')break;//拷贝完成了.
//      len++;
//      str++;
//  }
//  return len;
//}
////m^n 函数
////返回值:m^n 次方
//u32 usmart_pow(u8 m,u8 n)
//{
//  u32 result=1;
//  while(n--)result*=m;
//  return result;
//}
////把字符串转为数字
////支持 16 进制转换,但是 16 进制字母必须是大写的,且格式为以 0X 开头的.
////不支持负数
////*str:数字字符串指针
////*res:转换完的结果存放地址.
////返回值:0, 成功转换完成.其他,错误代码.
////1,数据格式错误.2,16 进制位数为 0.3,起始格式错误.4,十进制位数为 0.
//u8 usmart_str2num(u8*str,u32 *res)
//{
//  u32 t;
//  u8 bnum=0;  //数字的位数
//  u8 *p;
//  u8 hexdec=10;//默认为十进制数据
```

```
//  p=str;
//  *res=0;//清零.
//  while(1)
//  {
//      if((*p<='9'&&*p>='0')||(*p<='F'&&*p>='A')||(*p=='X'&&bnum==1))//参数合法
//      {
//          if(*p>='A')hexdec=16;   //字符串中存在字母,为 16 进制格式.
//          bnum++;                 //位数增加.
//      }else if(*p=='\0')break;    //碰到结束符,退出.
//      else return 1;              //不全是十进制或者 16 进制数据.
//      p++;
//  }
//  p=str;              //重新定位到字符串开始的地址.
//  if(hexdec==16)      //16 进制数据
//  {
//      if(bnum<3)return 2;         //位数小于 3，直接退出.因为 0X 就占了 2 个,如果 0X 后面不跟数据,则该数据非法.
//      if(*p=='0' && (*(p+1)=='X'))//必须以'0X'开头.
//      {
//          p+=2;   //偏移到数据起始地址.
//          bnum-=2;//减去偏移量
//      }else return 3;//起始头的格式不对
//  }else if(bnum==0)return 4;//位数为 0，直接退出.
//  while(1)
//  {
//      if(bnum)bnum--;
//      if(*p<='9'&&*p>='0')t=*p-'0';   //得到数字的值
//      else t=*p-'A'+10;               //得到 A~F 对应的值
//      *res+=t*usmart_pow(hexdec,bnum);
//      p++;
//      if(*p=='\0')break;//数据都查完了.
//  }
//  return 0;//成功转换
//}
////得到指令名
////*str:源字符串
////*cmdname:指令名
////*nlen:指令名长度
////maxlen:最大长度(做限制,指令不可能太长的)
////返回值:0,成功;其他,失败.
//u8 usmart_get_cmdname(u8*str,u8*cmdname,u8 *nlen,u8 maxlen)
//{
//  *nlen=0;
//  while(*str!=' '&&*str!='\0') //找到空格或者结束符则认为结束了
//  {
//      *cmdname=*str;
//      str++;
//      cmdname++;
//      (*nlen)++;//统计命令长度
//      if(*nlen>=maxlen)return 1;//错误的指令
//  }
//  *cmdname='\0';//加入结束符
//  return 0;//正常返回
//}
////获取下一个字符（当中间有很多空格的时候，此函数直接忽略空格，找到空格之后的第一个字符）
////str:字符串指针
```

```
////返回值:下一个字符
//u8 usmart_search_nextc(u8* str)
//{
//  str++;
//  while(*str==' '&&str!='\0')str++;
//  return *str;
//}
////从 str 中得到函数名
////*str:源字符串指针
////*fname:获取到的函数名字指针
////*pnum:函数的参数个数
////*rval:是否需要显示返回值(0,不需要;1,需要)
////返回值:0,成功;其他,错误代码.
//u8 usmart_get_fname(u8*str,u8*fname,u8 *pnum,u8 *rval)
//{
//  u8 res;
//  u8 fover=0;   //括号深度
//  u8 *strtemp;
//  u8 offset=0;
//  u8 parmnum=0;
//  u8 temp=1;
//  u8 fpname[6];//void+X+'/0'
//  u8 fplcnt=0; //第一个参数的长度计数器
//  u8 pcnt=0;   //参数计数器
//  u8 nchar;
//  //判断函数是否有返回值
//  strtemp=str;
//  while(*strtemp!='\0')//没有结束
//  {
//      if(*strtemp!=' '&&(pcnt&0X7F)<5)//最多记录 5 个字符
//      {
//          if(pcnt==0)pcnt|=0X80;//置位最高位,标记开始接收返回值类型
//          if(((pcnt&0x7f)==4)&&(*strtemp!='*'))break;//最后一个字符,必须是*
//          fpname[pcnt&0x7f]=*strtemp;//记录函数的返回值类型
//          pcnt++;
//      }else if(pcnt==0X85)break;
//      strtemp++;
//  }
//  if(pcnt)//接收完了
//  {
//      fpname[pcnt&0x7f]='\0';//加入结束符
//      if(usmart_strcmp(fpname,"void")==0)*rval=0;//不需要返回值
//      else *rval=1;                              //需要返回值
//      pcnt=0;
//  }
//  res=0;
//  strtemp=str;
//  while(*strtemp!='('&&*strtemp!='\0') //此代码找到函数名的真正起始位置
//  {
//      strtemp++;
//      res++;
//      if(*strtemp==' '||*strtemp=='*')
//      {
//          nchar=usmart_search_nextc(strtemp);     //获取下一个字符
//          if(nchar!='('&&nchar!='*')offset=res;   //跳过空格和*号
```

```
//      }
//  }
//  strtemp=str;
//  if(offset)strtemp+=offset+1;//跳到函数名开始的地方
//  res=0;
//  nchar=0;//是否正在字符串里面的标志,0，不在字符串;1，在字符串;
//  while(1)
//  {
//      if(*strtemp==0)
//      {
//          res=USMART_FUNCERR;//函数错误
//          break;
//      }else if(*strtemp=='('&&nchar==0)fover++;//括号深度增加一级
//      else if(*strtemp==')'&&nchar==0)
//      {
//          if(fover)fover--;
//          else res=USMART_FUNCERR;//错误结束,没收到'('
//          if(fover==0)break;//到末尾了,退出
//      }else if(*strtemp=='"')nchar=!nchar;

//      if(fover==0)//函数名还没接收完
//      {
//          if(*strtemp!=' ')//空格不属于函数名
//          {
//              *fname=*strtemp;//得到函数名
//              fname++;
//          }
//      }else //已经接受完了函数名了.
//      {
//          if(*strtemp==',')
//          {
//              temp=1;     //使能增加一个参数
//              pcnt++;
//          }else if(*strtemp!=' '&&*strtemp!='(')
//          {
//              if(pcnt==0&&fplcnt<5)       //当第一个参数来时,为了避免统计 void 类型的参数,必须做判断.
//              {
//                  fpname[fplcnt]=*strtemp;//记录参数特征.
//                  fplcnt++;
//              }
//              temp++; //得到有效参数(非空格)
//          }
//          if(fover==1&&temp==2)
//          {
//              temp++;     //防止重复增加
//              parmnum++;  //参数增加一个
//          }
//      }
//      strtemp++;
//  }
//  if(parmnum==1)//只有 1 个参数.
//  {
//      fpname[fplcnt]='\0';//加入结束符
//      if(usmart_strcmp(fpname,"void")==0)parmnum=0;//参数为 void,表示没有参数.
//  }
```

```
//  *pnum=parmnum;  //记录参数个数
//  *fname='\0';    //加入结束符
//  return res;     //返回执行结果
//}


////从 str 中得到一个函数的参数
////*str:源字符串指针
////*fparm:参数字符串指针
////*ptype:参数类型 0, 数字;1, 字符串;0XFF, 参数错误
////返回值:0,已经无参数了;其他,下一个参数的偏移量.
//u8 usmart_get_aparm(u8 *str,u8 *fparm,u8 *ptype)
//{
//  u8 i=0;
//  u8 enout=0;
//  u8 type=0;//默认是数字
//  u8 string=0; //标记 str 是否正在读
//  while(1)
//  {
//      if(*str==','&& string==0)enout=1;           //暂缓立即退出,目的是寻找下一个参数的起始地址
//      if((*str==')'||*str=='\0')&&string==0)break;//立即退出标识符
//      if(type==0)//默认是数字的
//      {
//          if((*str>='0' && *str<='9')||(*str>='a' && *str<='f')||(*str>='A' && *str<='F')||*str=='X'||*str=='x')//数字串检测
//          {
//              if(enout)break;                 //找到了下一个参数,直接退出.
//              if(*str>='a')*fparm=*str-0X20;  //小写转换为大写
//              else *fparm=*str;               //小写或者数字保持不变
//              fparm++;
//          }else if(*str=='"')//找到字符串的开始标志
//          {
//              if(enout)break;//找到,后才找到",认为结束了.
//              type=1;
//              string=1;//登记 STRING 正在读了
//          }else if(*str!=' '&&*str!=',')//发现非法字符,参数错误
//          {
//              type=0XFF;
//              break;
//          }
//      }else//string 类
//      {
//          if(*str=='"')string=0;
//          if(enout)break;         //找到了下一个参数,直接退出.
//          if(string)              //字符串正在读
//          {
//              if(*str=='\\')      //遇到转义符(不复制转义符)
//              {
//                  str++;          //偏移到转义符后面的字符,不管什么字符,直接 COPY
//                  i++;
//              }
//              *fparm=*str;        //小写或者数字保持不变
//              fparm++;
//          }
//      }
//      i++;//偏移量增加
```

```
//      str++;
//  }
//  *fparm='\0';    //加入结束符
//  *ptype=type;    //返回参数类型
//  return i;       //返回参数长度
//}
////得到指定参数的起始地址
////num:第 num 个参数,范围 0~9.
////返回值:该参数的起始地址
//u8 usmart_get_parmpos(u8 num)
//{
//  u8 temp=0;
//  u8 i;
//  for(i=0;i<num;i++)temp+=usmart_dev.plentbl[i];
//  return temp;
//}
////从 str 中得到函数参数
////str:源字符串;
////parn:参数的多少.0 表示无参数 void 类型
////返回值:0,成功;其他,错误代码.
//u8 usmart_get_fparam(u8*str,u8 *parn)
//{
//  u8 i,type;
//  u32 res;
//  u8 n=0;
//  u8 len;
//  u8 tstr[PARM_LEN+1];//字节长度的缓存,最多可以存放 PARM_LEN 个字符的字符串
//  for(i=0;i<MAX_PARM;i++)usmart_dev.plentbl[i]=0;//清空参数长度表
//  while(*str!='(')//偏移到参数开始的地方
//  {
//      str++;
//      if(*str=='\0')return USMART_FUNCERR;//遇到结束符了
//  }
//  str++;//偏移到"("之后的第一个字节
//  while(1)
//  {
//      i=usmart_get_aparm(str,tstr,&type); //得到第一个参数
//      str+=i;                             //偏移
//      switch(type)
//      {
//          case 0: //数字
//              if(tstr[0]!='\0')               //接收到的参数有效
//              {
//                  i=usmart_str2num(tstr,&res);    //记录该参数
//                  if(i)return USMART_PARMERR;     //参数错误.
//                  *(u32*)(usmart_dev.parm+usmart_get_parmpos(n))=res;//记录转换成功的结果.
//                  usmart_dev.parmtype&=~(1<<n);   //标记数字
//                  usmart_dev.plentbl[n]=4;        //该参数的长度为 4
//                  n++;                            //参数增加
//                  if(n>MAX_PARM)return USMART_PARMOVER;//参数太多
//              }
//              break;
//          case 1://字符串
//              len=usmart_strlen(tstr)+1;  //包含了结束符'\0'
//              usmart_strcopy(tstr,&usmart_dev.parm[usmart_get_parmpos(n)]);//拷贝 tstr 数据到 usmart_dev.parm[n]
```

```
//              usmart_dev.parmtype|=1<<n;  //标记字符串
//              usmart_dev.plentbl[n]=len;  //该参数的长度为 len
//              n++;
//              if(n>MAX_PARM)return USMART_PARMOVER;//参数太多
//              break;
//          case 0XFF://错误
//              return USMART_PARMERR;//参数错误
//      }
//      if(*str==')'||*str=='\0')break;//查到结束标志了.
//  }
//  *parn=n;    //记录参数的个数
//  return USMART_OK;//正确得到了参数
//}
```

```
#ifndef __USMART_STR_H
#define __USMART_STR_H
#include "n32g45x.h"
//////////////////////////////////////////////////////////////////////////////////
//本程序只供学习使用，未经作者许可，不得用于其它任何用途
//ALIENTEK STM32 开发板
//正点原子@ALIENTEK
//技术论坛:www.openedv.com
//版本：V3.1
//版权所有，盗版必究。
//Copyright(C) 正点原子 2011-2021
//All rights reserved
//********************************************************************************
//升级说明
//V1.4
//增加了对参数为 string 类型的函数的支持.适用范围大大提高.
//优化了内存占用,静态内存占用为 79 个字节@10 个参数.动态适应数字及字符串长度
//V2.0
//1,修改了 list 指令,打印函数的完整表达式.
//2,增加了 id 指令,打印每个函数的入口地址.
//3,修改了参数匹配,支持函数参数的调用(输入入口地址).
//4,增加了函数名长度宏定义.
//V2.1 20110707
//1,增加 dec,hex 两个指令,用于设置参数显示进制,及执行进制转换.
//注:当 dec,hex 不带参数的时候,即设定显示参数进制.当后跟参数的时候,即执行进制转换.
//如:"dec 0XFF" 则会将 0XFF 转为 255,由串口返回.
//如:"hex 100"   则会将 100 转为 0X64,由串口返回
//2,新增 usmart_get_cmdname 函数,用于获取指令名字.
```

```
//V2.2 20110726
//1,修正了 void 类型参数的参数统计错误.
//2,修改数据显示格式默认为 16 进制.
//V2.3 20110815
//1,去掉了函数名后必须跟"("的限制.
//2,修正了字符串参数中不能有"("的 bug.
//3,修改了函数默认显示参数格式的修改方式.
//V2.4 20110905
//1,修改了 usmart_get_cmdname 函数,增加最大参数长度限制.避免了输入错误参数时的死机现象.
//2,增加 USMART_ENTIM2_SCAN 宏定义,用于配置是否使用 TIM2 定时执行 scan 函数.
//V2.5 20110930
//1,修改 usmart_init 函数为 void usmart_init(u8 sysclk),可以根据系统频率自动设定扫描时间.(固定 100ms)
//2,去掉了 usmart_init 函数中的 uart_init 函数,串口初始化必须在外部初始化,方便用户自行管理.
//V2.6 20111009
//1,增加了 read_addr 和 write_addr 两个函数.可以利用这两个函数读写内部任意地址(必须是有效地址).更加方便调试.
//2,read_addr 和 write_addr 两个函数可以通过设置 USMART_USE_WRFUNS 为来使能和关闭.
//3,修改了 usmart_strcmp,使其规范化.
//V2.7 20111024
//1,修正了返回值 16 进制显示时不换行的 bug.
//2,增加了函数是否有返回值的判断,如果没有返回值,则不会显示.有返回值时才显示其返回值.
//V2.8 20111116
//1,修正了 list 等不带参数的指令发送后可能导致死机的 bug.
//V2.9 20120917
//1,修改了形如：void*xxx(void)类型函数不能识别的 bug。
//V3.0 20130425
//1,新增了字符串参数对转义符的支持。
//V3.1 20131120
//1,增加 runtime 系统指令,可以用于统计函数执行时间.
//用法:
//发送:runtime 1 ,则开启函数执行时间统计功能
//发送:runtime 0 ,则关闭函数执行时间统计功能
///runtime 统计功能,必须设置:USMART_ENTIMX_SCAN 为 1,才可以使用!!
////////////////////////////////////////////////////////////////////////////////


u8 usmart_get_parmpos(u8 num);                      //得到某个参数在参数列里面的起始位置
u8 usmart_strcmp(u8*str1,u8 *str2);                 //对比两个字符串是否相等
u32 usmart_pow(u8 m,u8 n);                          //M^N 次方
u8 usmart_str2num(u8*str,u32 *res);                 //字符串转为数字
u8 usmart_get_cmdname(u8*str,u8*cmdname,u8 *nlen,u8 maxlen);//从 str 中得到指令名,并返回指令长度
u8 usmart_get_fname(u8*str,u8*fname,u8 *pnum,u8 *rval);     //从 str 中得到函数名
u8 usmart_get_aparm(u8 *str,u8 *fparm,u8 *ptype);   //从 str 中得到一个函数参数
u8 usmart_get_fparam(u8*str,u8 *parn);              //得到 str 中所有的函数参数.
#endif
```